

Chapter 14: Indexing

Outline

Basic Concepts

Ordered Indices

B⁺-Tree Index

B+-Tree File Organization

B-Tree Index Files

Indices on Multiple Keys

Indexing on Flash

Indexing in Main Memory

Write Optimized Indices

- Log Structured Merge (LSM) Tree

- Buffer Tree

Bitmap Indices

Basic Concepts

Indexing mechanisms used **to speed up access** to desired data.

E.g., author catalog in library

Search Key - attribute to set of attributes used to look up records in a file.

An **index file** consists of records (called **index entries**) of the form

search-key	pointer
------------	---------

Index files are typically much smaller than the original file

Two basic kinds of indices:

Ordered indices: search keys are stored in **sorted order**

Hash indices: search keys are distributed uniformly across “buckets” using a “hash function”.

Index Evaluation Metrics

Access types supported efficiently. E.g.,

Point query: records with a specified value in the attribute

Range query: or records with an attribute value falling in a specified range of values.

Access time

Insertion time

Deletion time

Space overhead

Ordered Indices

In an ordered index, index entries are stored sorted on the search key value. E.g., author catalog in library.

- **Primary index(主索引)**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.

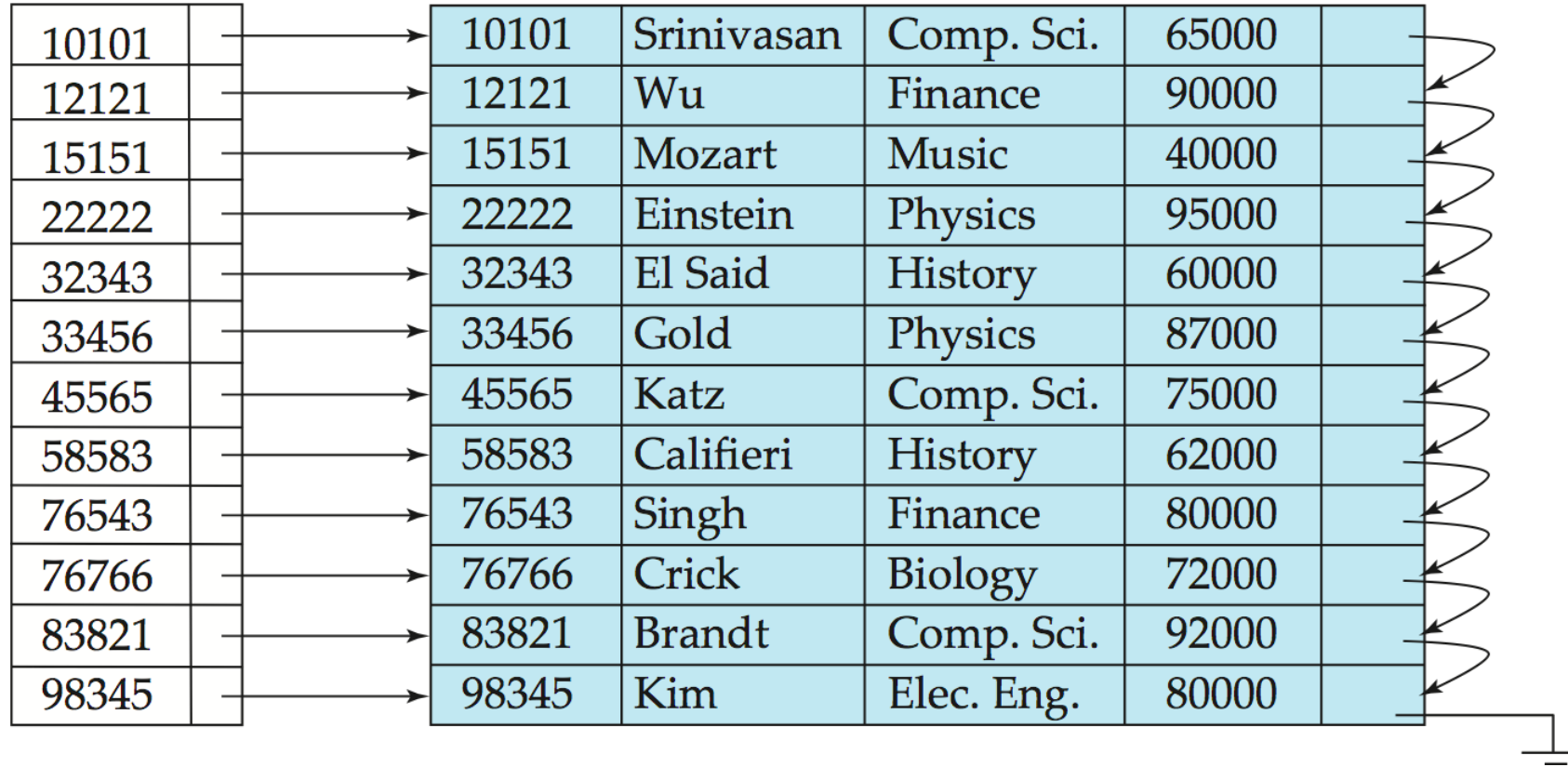
Also called **clustering index(聚集索引)**

The search key of a primary index is usually but not necessarily the primary key.

- **Secondary index(辅助索引)**: an index whose search key specifies an order different from the sequential order of the file. Also called **non-clustering index**.

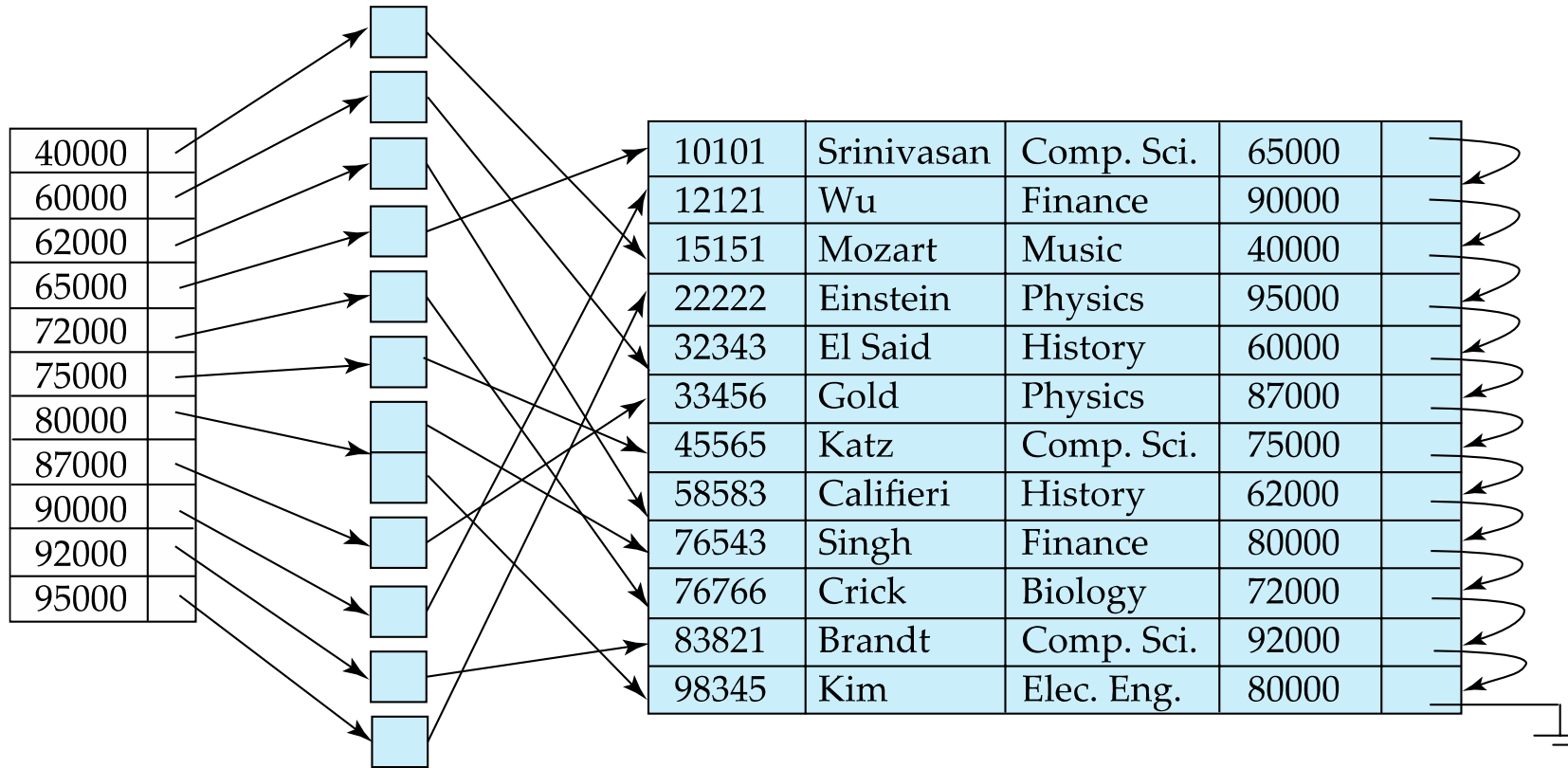
Index-sequential file(索引顺序文件): ordered sequential file with a primary index.

Primary index(主索引)



primary index on **ID** attribute of *instructor*

Secondary Indices Example

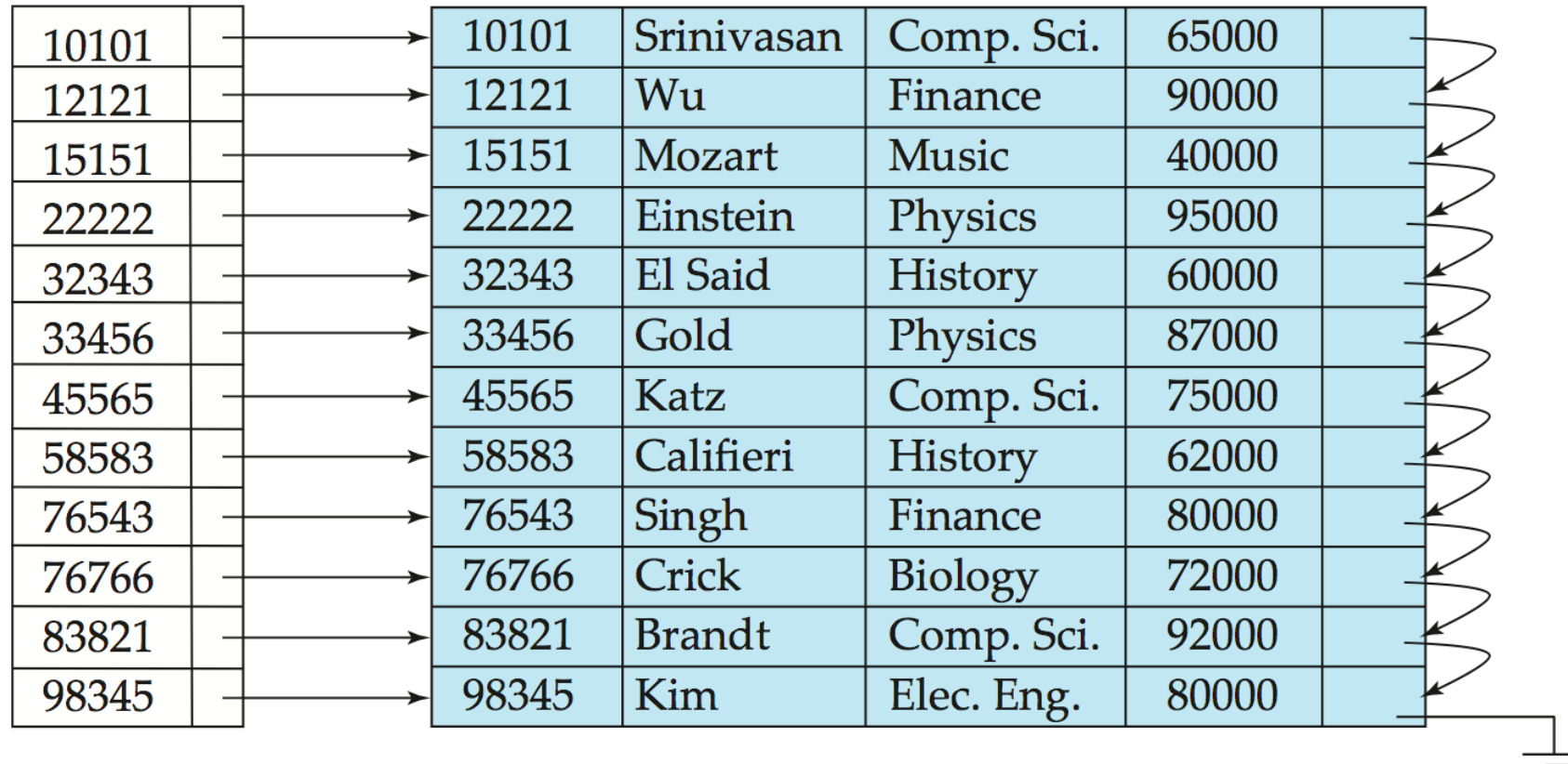


Secondary index on *salary* attribute of *instructor*

Dense Index Files

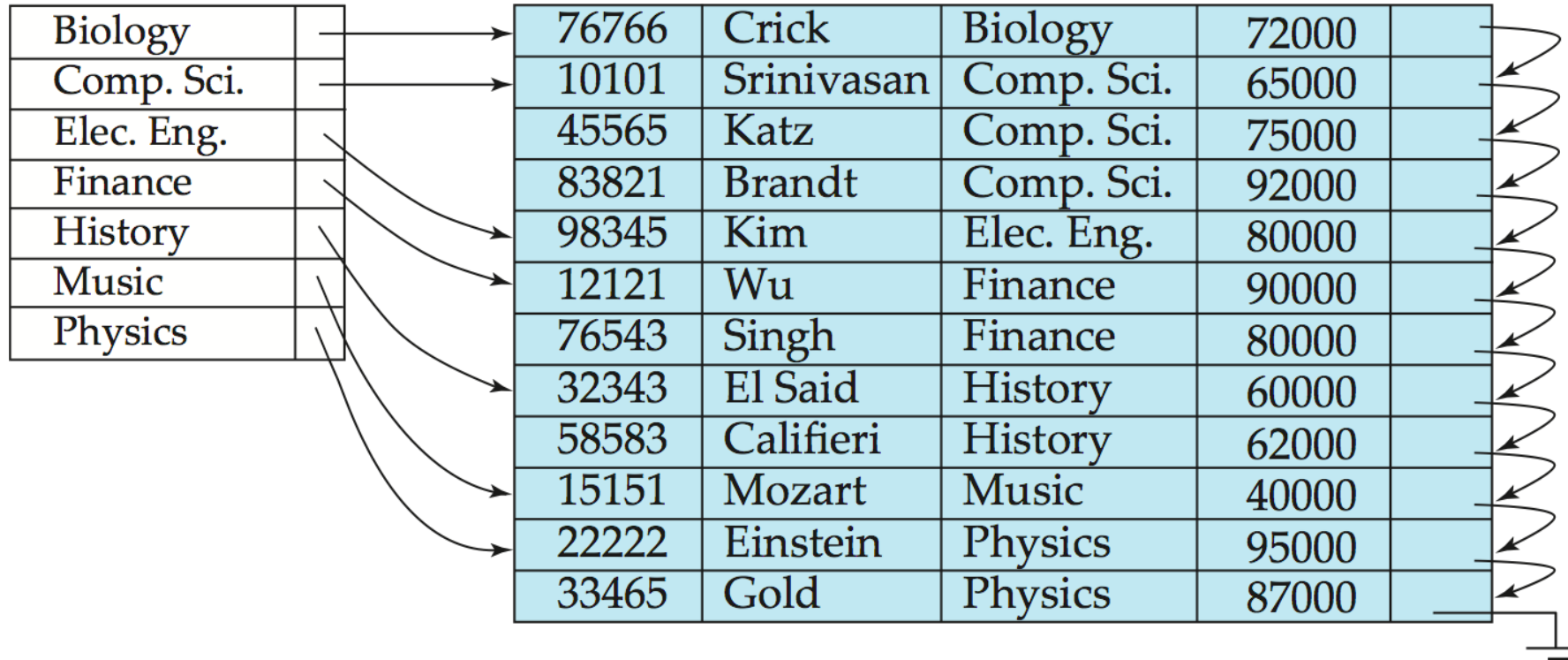
Dense index(稠密索引) — Index record appears for **every** search-key value in the file.

E.g. index on *ID* attribute of *instructor* relation



Dense Index Files (Cont.)

Dense index on *dept_name*, with *instructor* file sorted on *dept_name*



Sparse Index Files

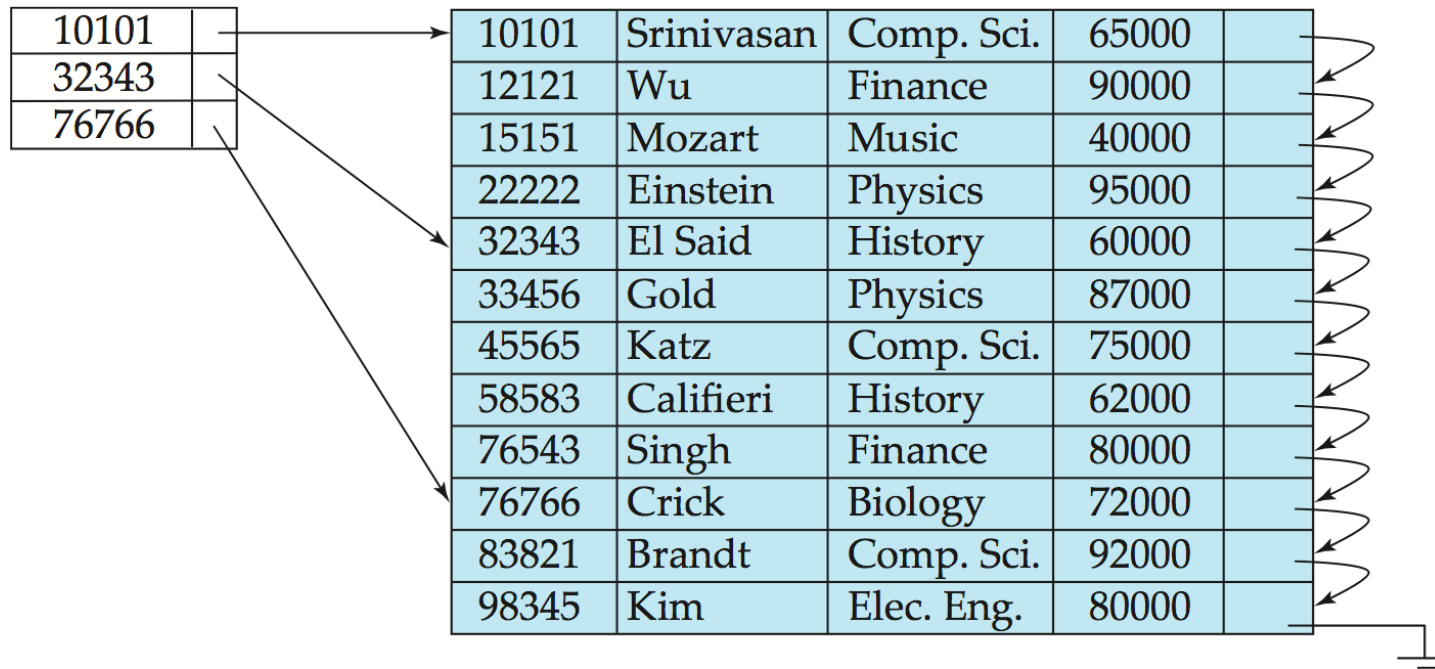
Sparse Index(稀疏索引): contains index records for only **some** search-key values.

Applicable when records are sequentially ordered on search-key

To locate a record with search-key value K we:

Find index record with largest search-key value $< K$

Search file sequentially starting at the record to which the index record points



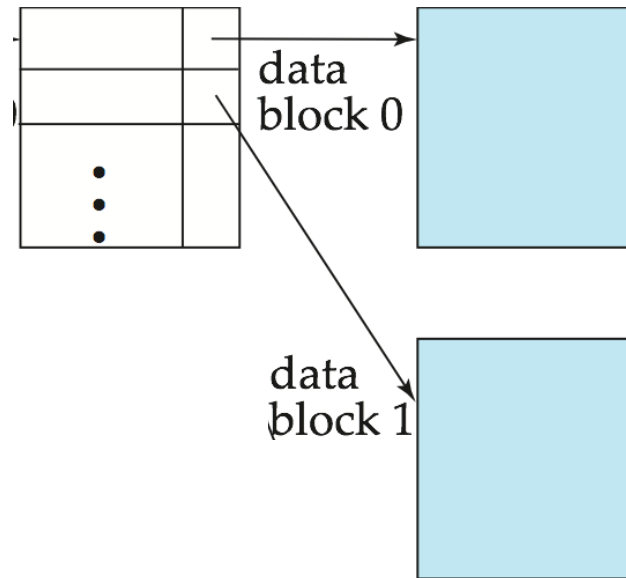
Sparse Index Files (Cont.)

Compared to dense indices:

Less space and less maintenance overhead for insertions and deletions.

Generally slower than dense index for locating records.

Good tradeoff: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.



Multilevel Index (多级索引)

If primary index does not fit in memory, access becomes expensive.

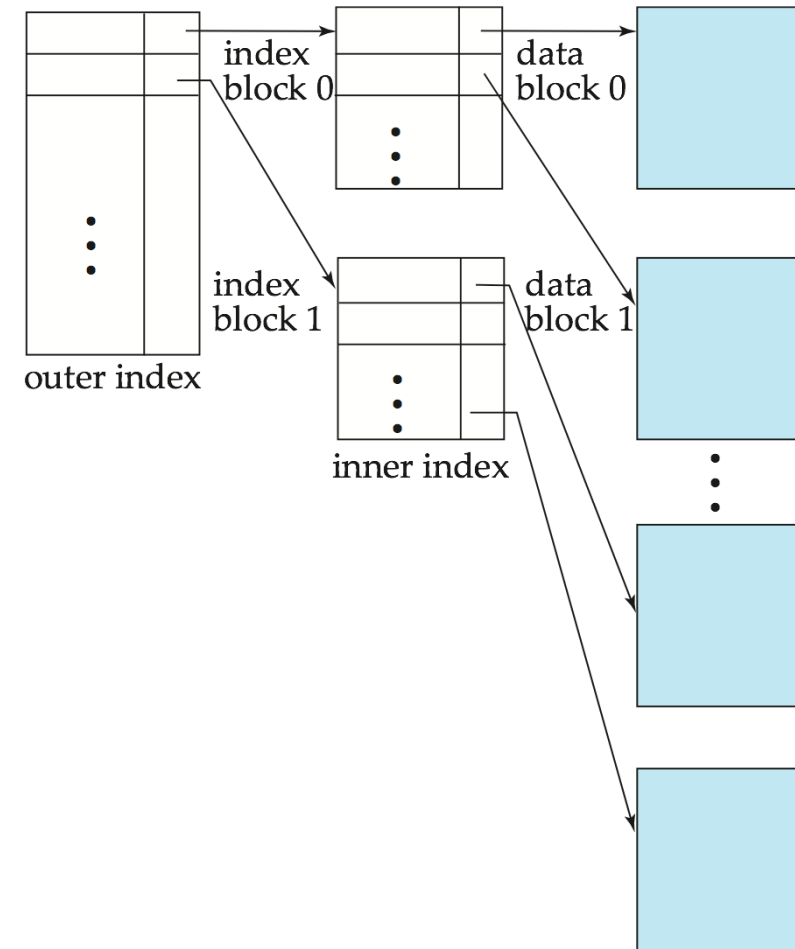
Solution: treat primary index kept on disk as a sequential file and construct a sparse index on it.

outer index – a sparse index of primary index

inner index – the primary index file

If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.

Indices at all levels must be updated on insertion or deletion from the file.



B⁺-Tree Index

A B⁺-tree is a rooted tree satisfying the following properties:

All paths from root to leaf are of the same length

Inner node(not a root or a leaf): between $\lceil n/2 \rceil$ and n children.

Leaf node: between $\lceil (n-1)/2 \rceil$ and $n-1$ values

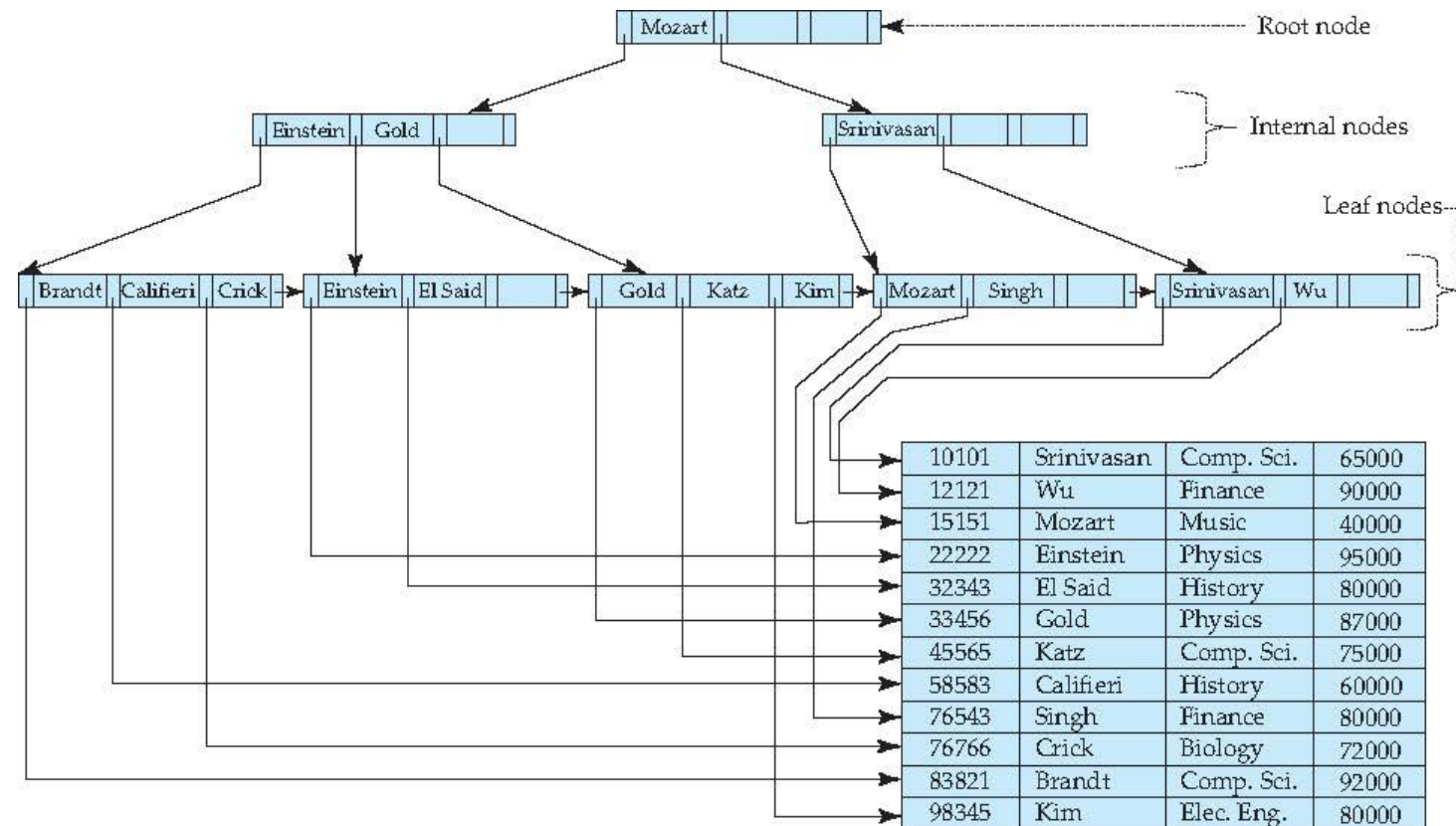
Special cases:

If the root is not a leaf:

at least 2 children.

If the root is a leaf :

between 0 and $(n-1)$ values.



B⁺-Tree Node Structure

Typical node

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

K_i are the search-key values

P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).

The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$

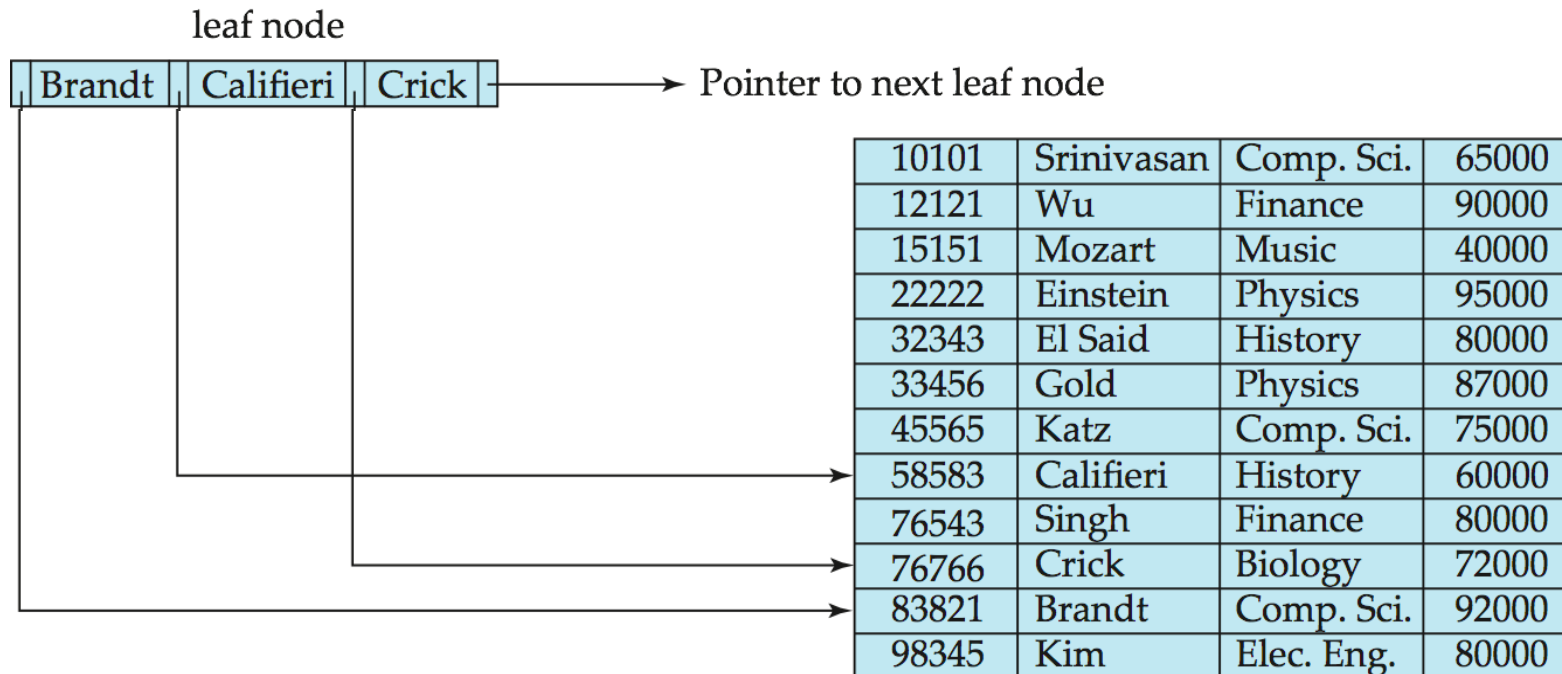
Leaf Nodes in B⁺-Trees

Properties of a leaf node:

For $i = 1, 2, \dots, n-1$, pointer P_i points to a file record with search-key value K_i ,

If L_i, L_j are leaf nodes and $i < j$, L_i 's search-key values are less than or equal to L_j 's search-key values

P_n points to next leaf node in search-key order



Non-Leaf Nodes in B⁺-Trees

Non leaf nodes form a multi-level sparse index on the leaf nodes.

For a non-leaf node with m pointers:

P_1 : all the search-keys in the subtree to which P_1 points are less than K_1

P_i ($2 \leq i \leq n - 1$): all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i

P_n : All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

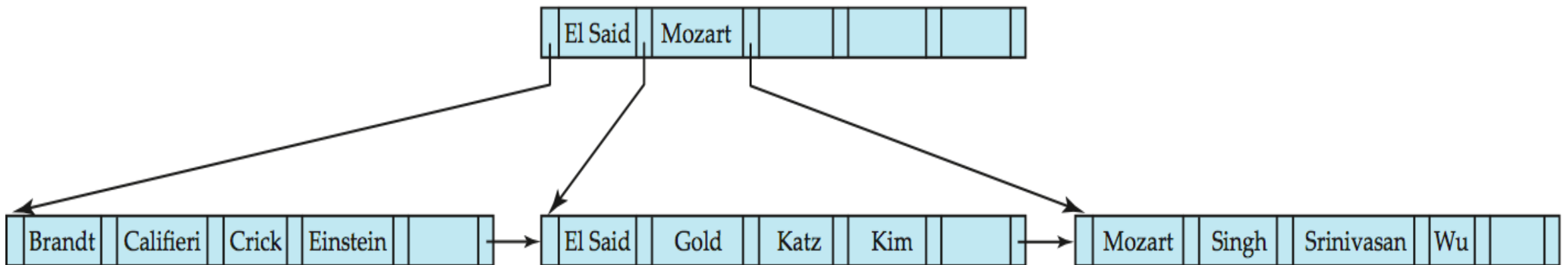
Example of B⁺-tree

B⁺-tree for *instructor* file ($n = 6$)

Leaf nodes must have between 3 and 5 values
($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).

Non-leaf nodes other than root must have between 3 and 6 children ($\lceil n/2 \rceil$ and n with $n = 6$).

Root must have at least 2 children.



Observations about B⁺-trees

Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.

The non-leaf levels of the B⁺-tree form a hierarchy of sparse indices.

The B⁺-tree contains a relatively small number of levels

- ▶ Level below root has at least $2 \cdot \lceil n/2 \rceil$ values
- ▶ Next level has at least $2 \cdot \lceil n/2 \rceil \cdot \lceil n/2 \rceil$ values
- ▶ .. etc.

If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil \sim \lceil \log_{\lceil n/2 \rceil}(K/2) \rceil + 1$

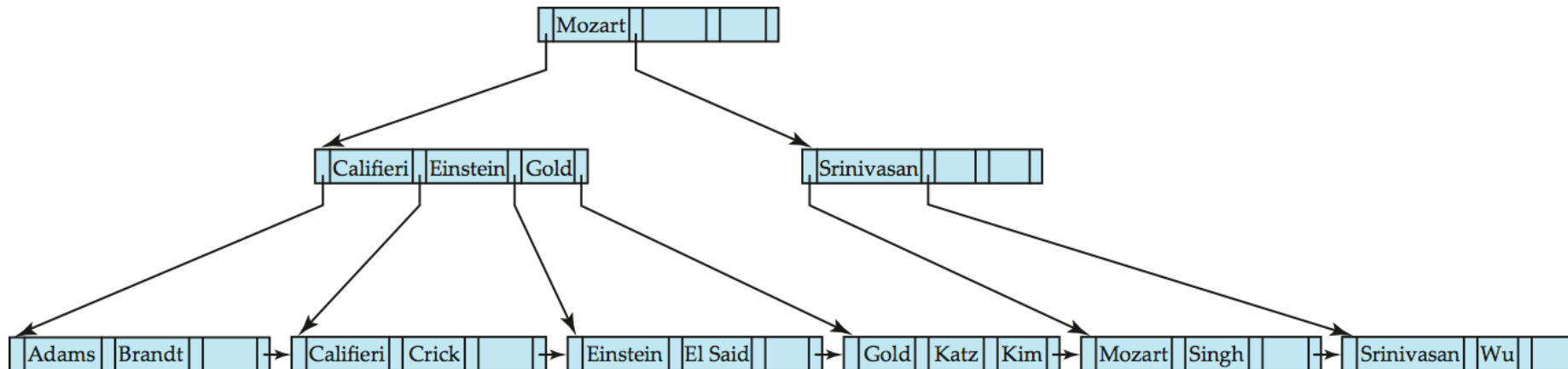
thus searches can be conducted efficiently.

Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time .

Query on B⁺-Tree

function *find(v)* // Find record with search-key value V

1. $C = \text{root}$
2. While C is not a leaf node {
 1. Let i be least value s.t. $V \leq K_i$.
 2. If no such exists, set $C = \text{last non-null pointer in } C$
 3. Else { if ($V = K_i$) Set $C = P_{i+1}$ else set $C = P_i$ }}
3. Let i be least value s.t. $K_i = V$
4. If there is such a value i , follow pointer P_i to the desired record.
5. Else no record with search-key value k exists.



Query on B⁺-Tree(Cont.)

If there are K search-key values in the file, the height of the tree is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.

A node is generally the same size as a disk block, typically 4 kilobytes and n is typically around 100 (40 bytes per index entry).

With 1 million search key values and $n = 100$

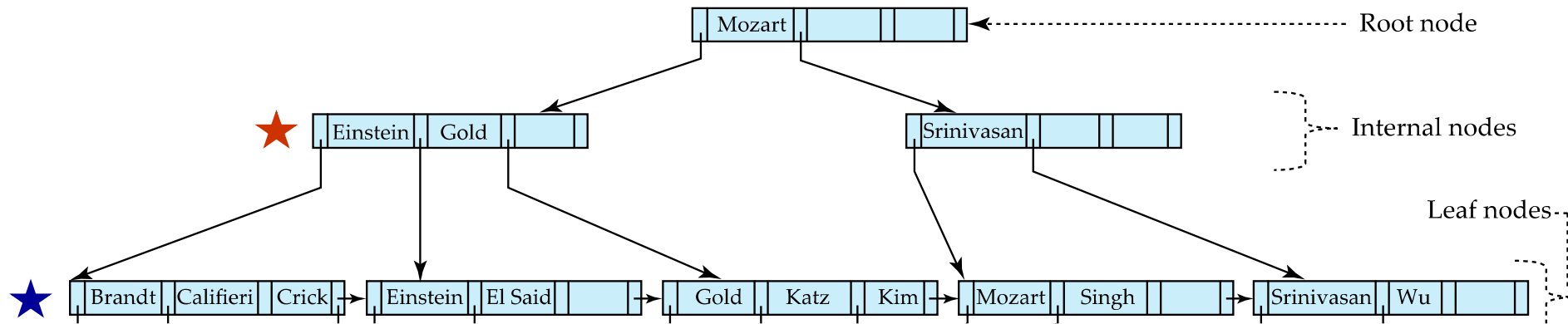
at most $\log_{50}(1,000,000) = 4$ nodes are accessed in a lookup.

Contrast this with a balanced binary tree with 1 million search key values — around 20 nodes are accessed in a lookup

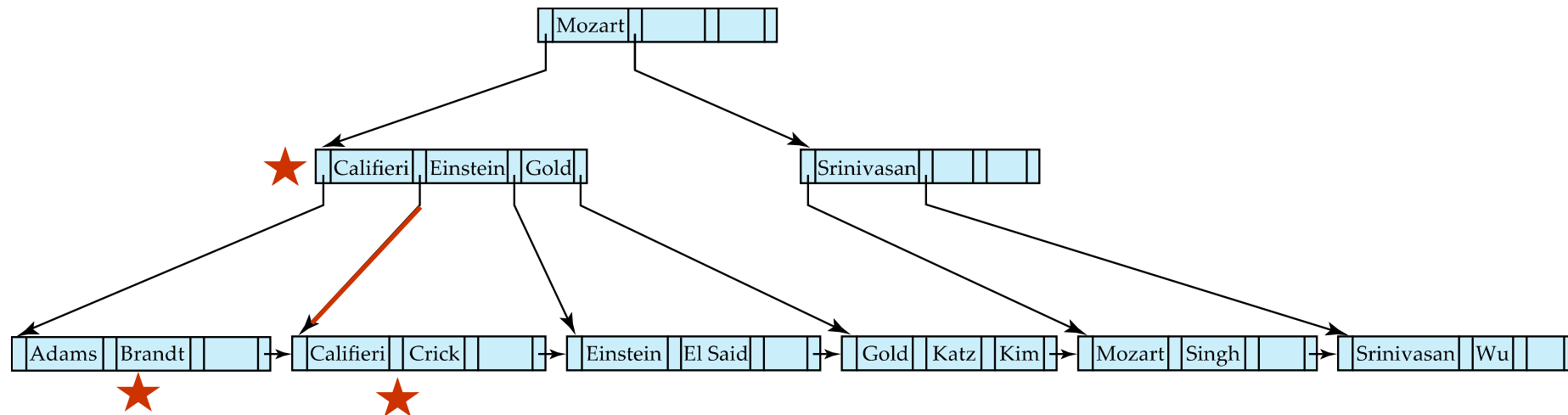
above difference is significant since every node access may need a disk I/O, costing around 20 milliseconds

Examples of Insert on B⁺-Tree

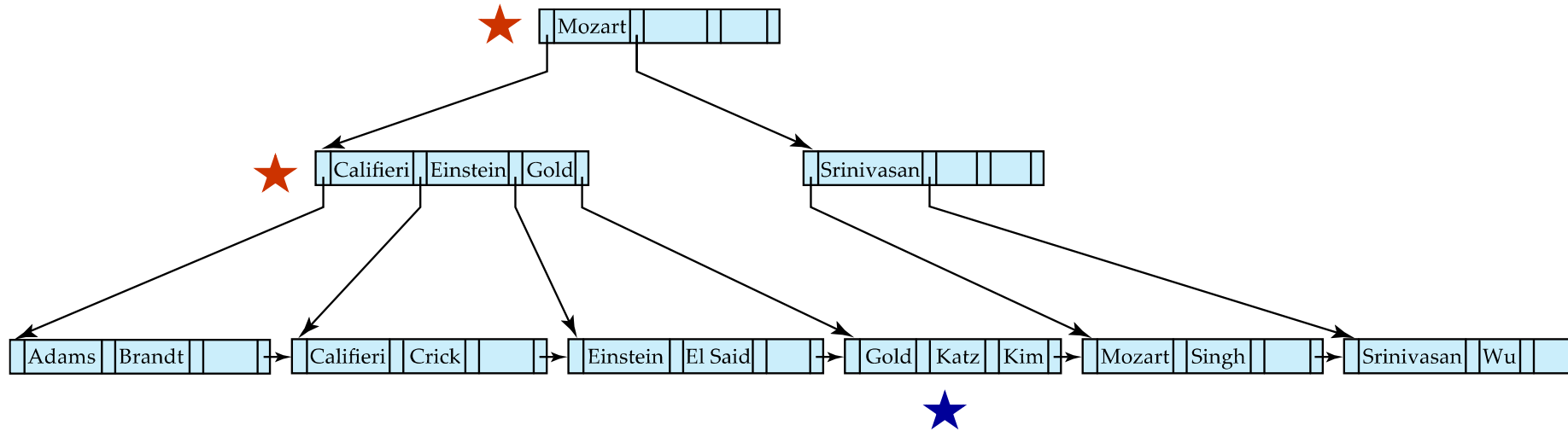
Affected nodes: ★



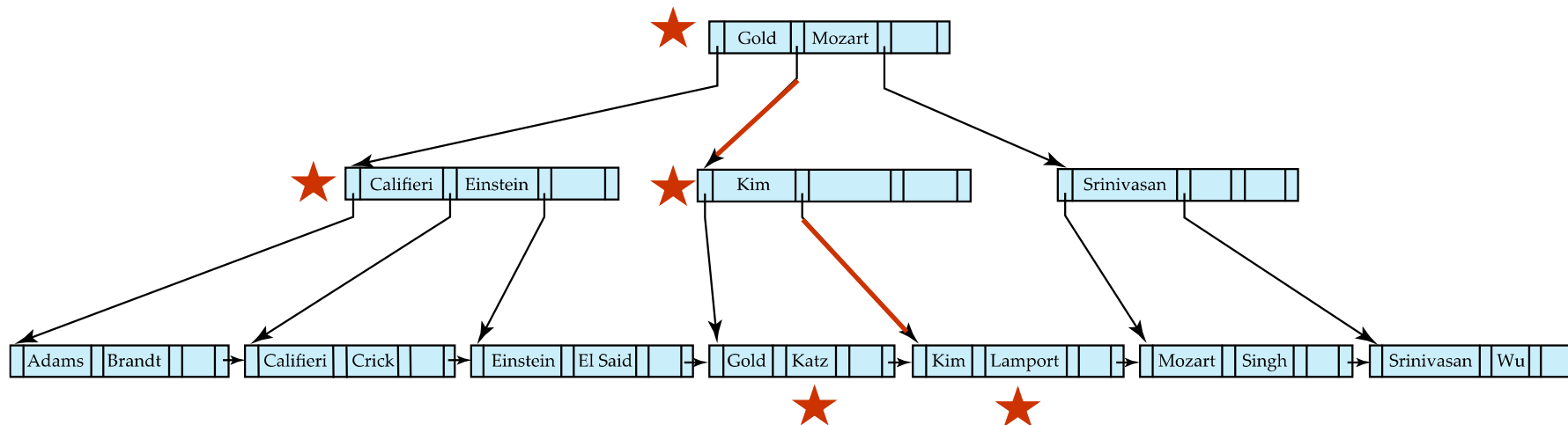
B⁺-Tree before and after insertion of “Adams”



Examples of Insert on B⁺-Tree



B⁺-Tree before and after insertion of “**Lampport**”



Insert on B⁺-Trees

function *insert* (*pr*, *v*)

/* Assume record already added to the file. Let

pr be pointer to the record, and let

v be the search key value of the record */

1. Find the leaf node in which the search-key value would appear
 1. **If there is room in the leaf node**, insert (*v*, *pr*) pair in the leaf node
 2. **Otherwise**, split the node (along with the new (*v*, *pr*) entry) as discussed in the next slide, and propagate updates to parent nodes.

Insert on B⁺-Trees (Cont.)

Splitting a leaf node:

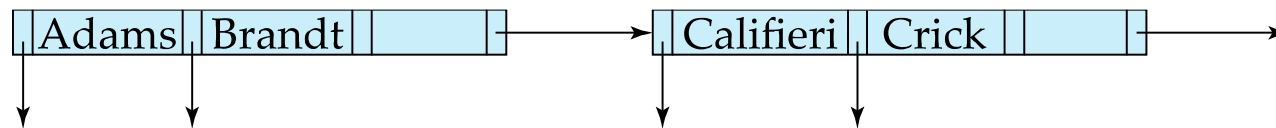
take the n (search-key value, pointer) pairs (including the one being inserted) in sorted order. Place the first $\lceil n/2 \rceil$ in the original node, and the rest in a new node.

let the new node be p , and let k be the least key value in p . Insert (k, p) in the parent of the node being split.

If the parent is full, split it and **propagate** the split further up.

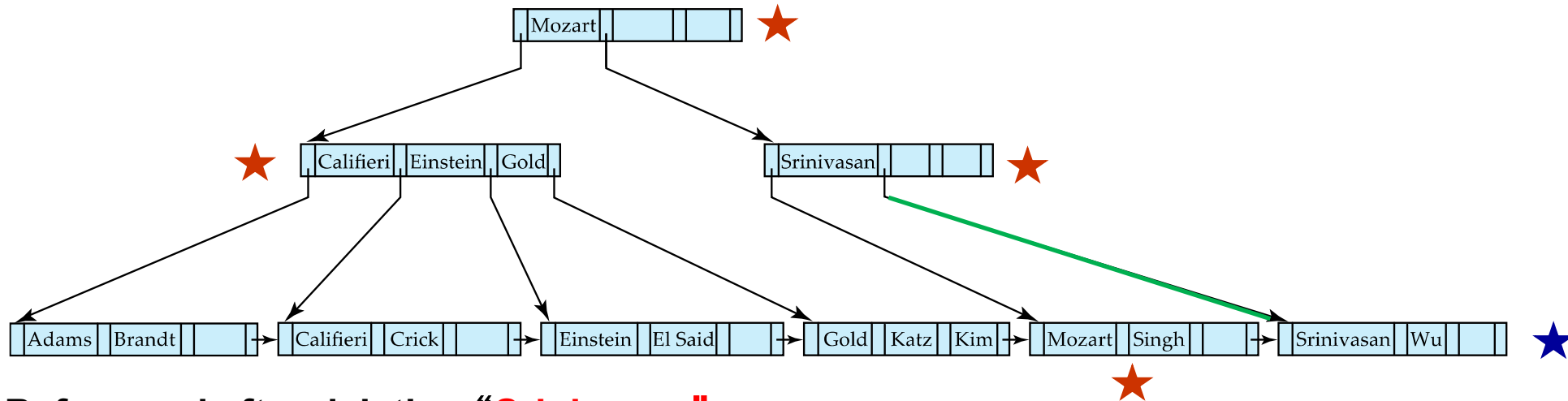
Splitting of nodes proceeds upwards till a node that is not full is found.

In the worst case the root node may be split increasing the height of the tree by 1.

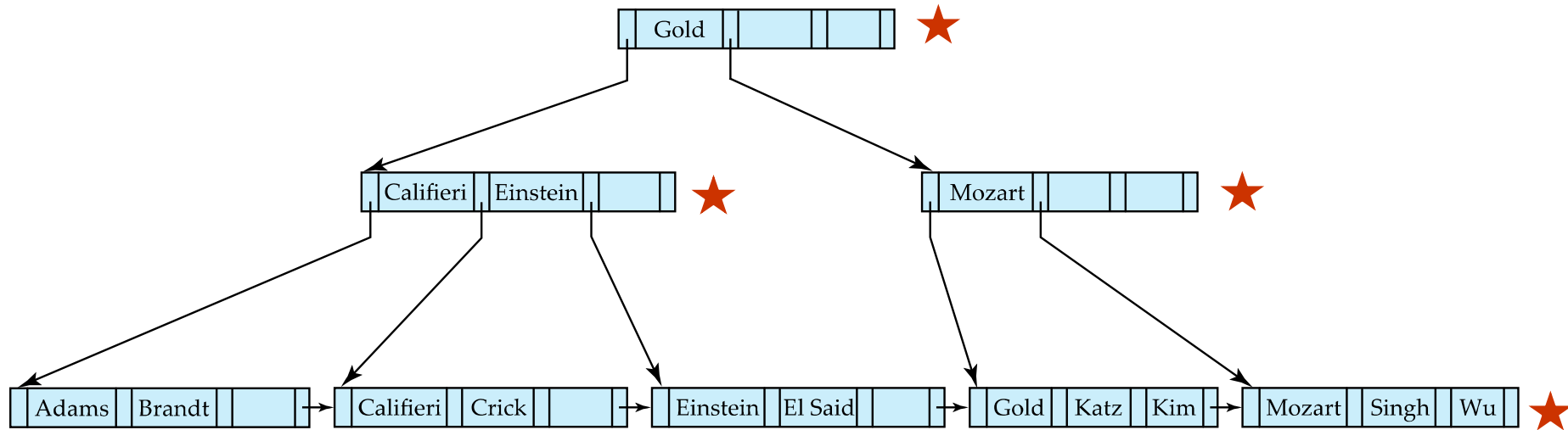


Result of splitting node containing Brandt, Califieri and Crick on inserting Adams
Next step: insert entry with (Califieri, pointer-to-new-node) into parent

Examples of Delete on B⁺-Tree

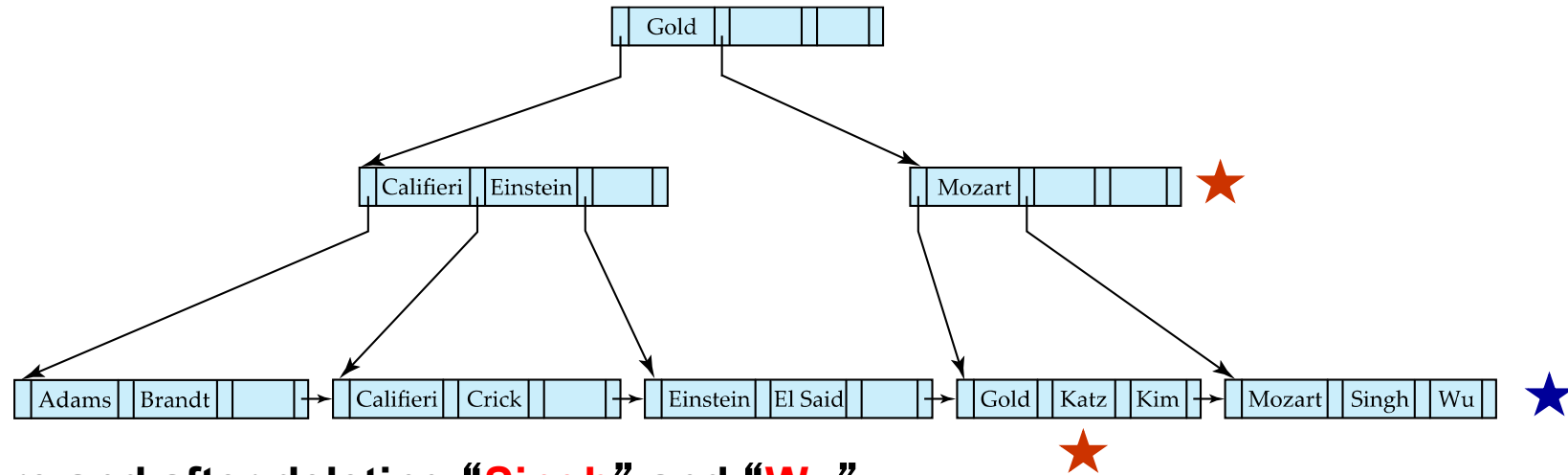


Before and after deleting **"Srinivasan"**

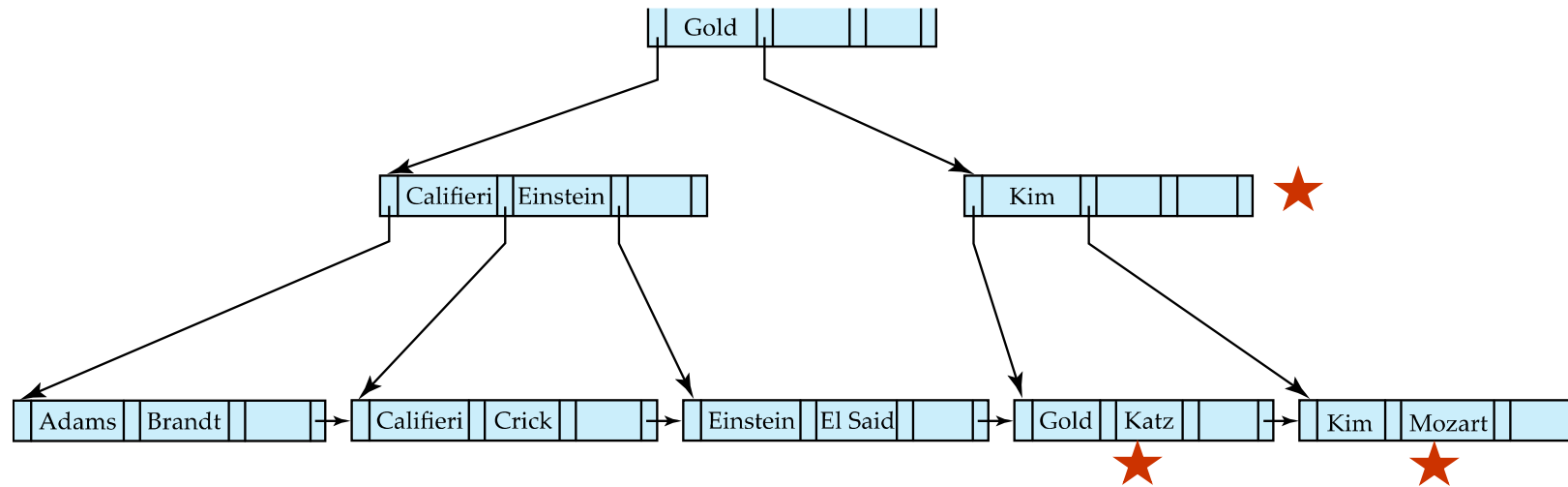


Deleting "Srinivasan" causes **merging** of under-full leaves

Examples of Delete on B⁺-Tree (Cont.)



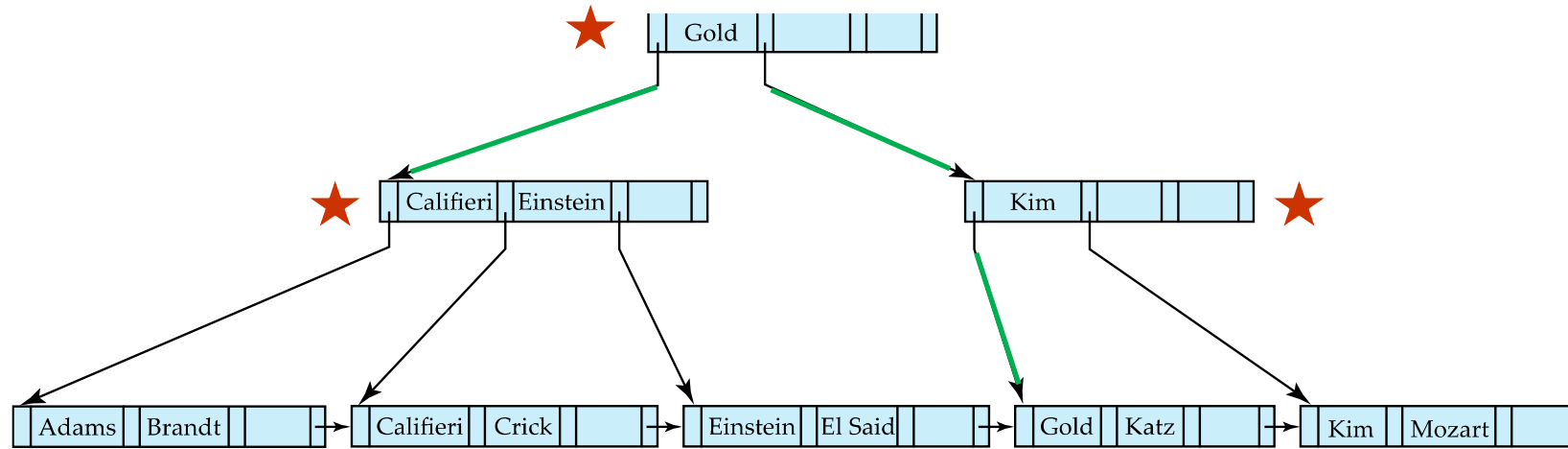
Before and after deleting “Singh” and “Wu”



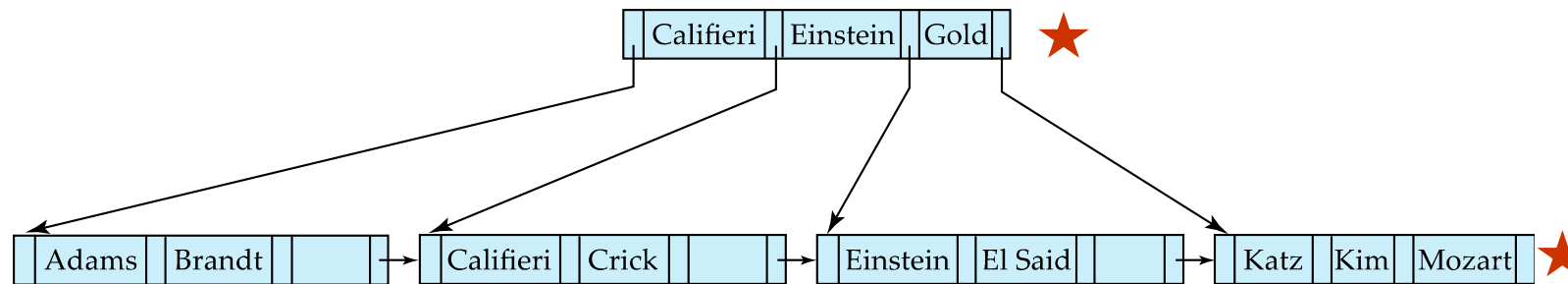
Leaf containing Singh and Wu became underfull, and **borrowed a value** Kim from its left sibling

Search-key value in the parent changes as a result

Example of Delete on B⁺-tree (Cont.)



Before and after deletion of “**Gold**”



Node with Gold and Katz became underfull, and was **merged** with its sibling

Parent node becomes underfull, and is **merged** with its sibling

Value separating two nodes (at the parent) is pulled down when merging

Root node then has only one child, and is **deleted**

Delete on B⁺-Tree

Function *delete*(*pr*, *v*)

/* Assume record already deleted from file. Let *v* be the search key value of the record, and *pr* be the pointer to the record. */

Remove (*pr*, *v*) from the leaf node

If the node has too few entries due to the removal, and the entries in the node and a sibling fit into a single node, then *merge siblings*:

Insert all the search-key values in the two nodes into a single node (the one on the left), and delete the other node.

Delete the pair (K_{i-1}, P_i) , where P_i is the pointer to the deleted node, from its parent, recursively using the above procedure.

Delete on B⁺-Tree(Cont.)

Otherwise, if the node has too few entries due to the removal, but the entries in the node and a sibling do not fit into a single node, then **redistribute pointers**:

Redistribute the pointers between the node and a sibling such that both have more than the minimum number of entries.

Update the corresponding search-key value in the parent of the node.

The node deletions may cascade upwards till a node which has $\lceil n/2 \rceil$ or more pointers is found.

If the root node has only one pointer after deletion, it is deleted and the sole child becomes the root.

Complexity of Updates

Cost (in terms of number of I/O operations) of insertion and deletion of a single entry proportional to height of the tree

With K entries and maximum fanout of n , worst case complexity of insert/delete of an entry is $O(\log_{\lceil n/2 \rceil}(K))$

In practice, number of I/O operations is less:

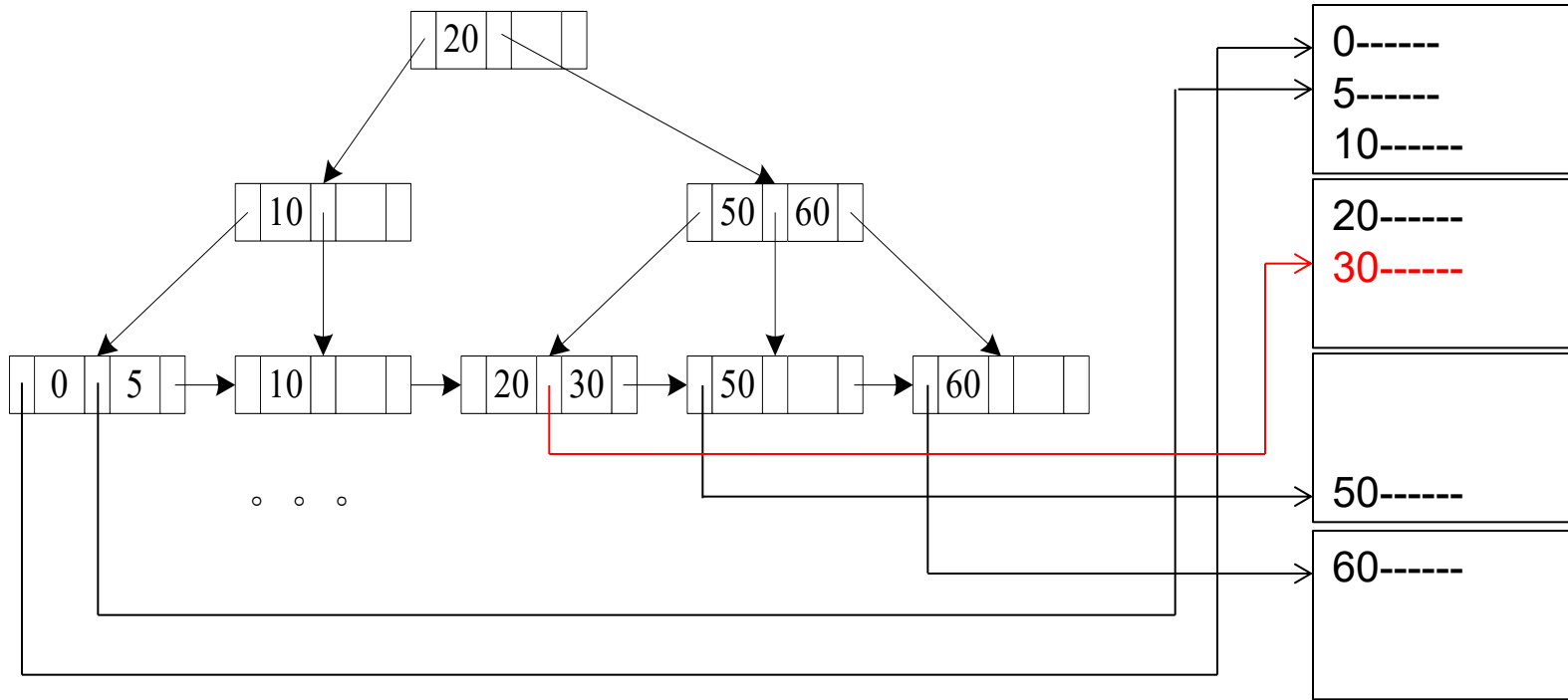
- Internal nodes tend to be in buffer

- Splits/merges are rare, most insert/delete operations only affect a leaf node

Average node occupancy depends on insertion order

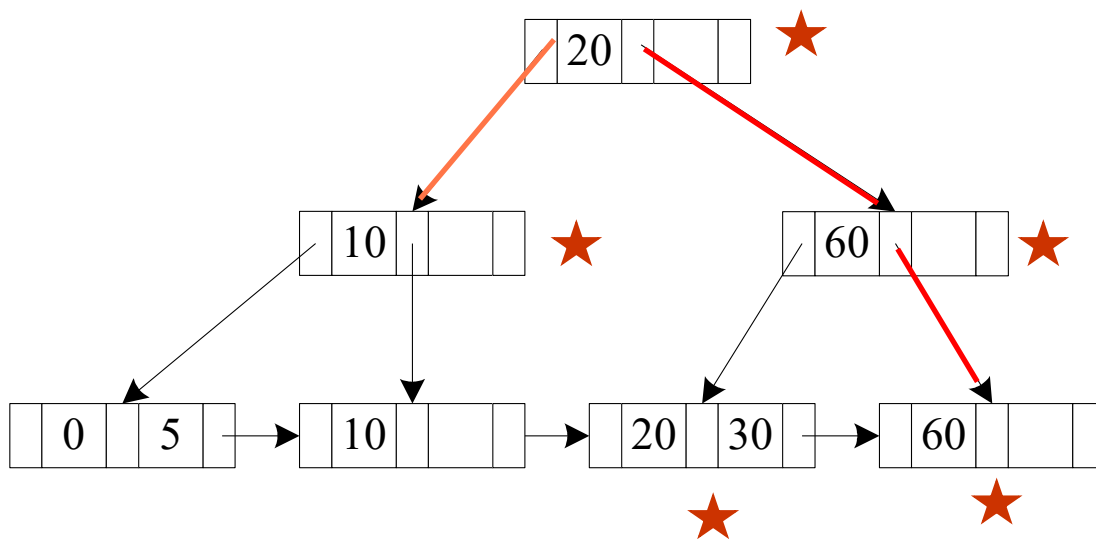
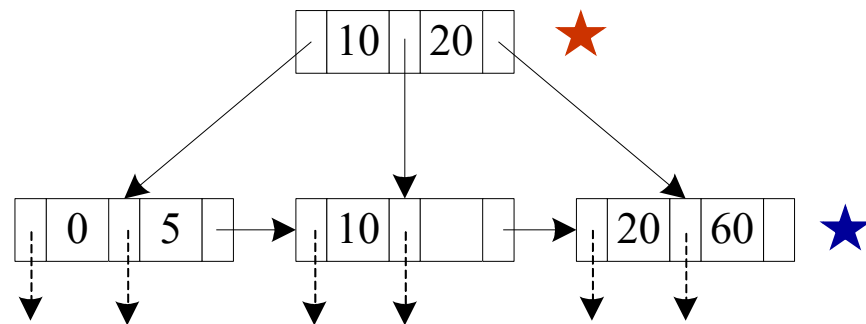
- $2/3$ with random, $1/2$ with insertion in sorted order

Example of B+ -Tree

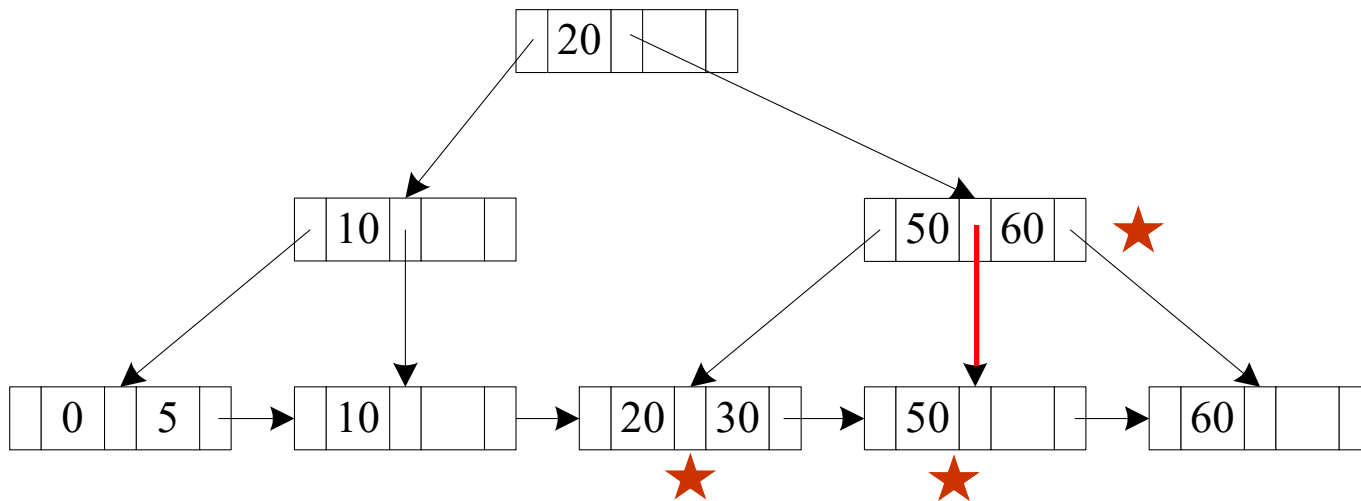
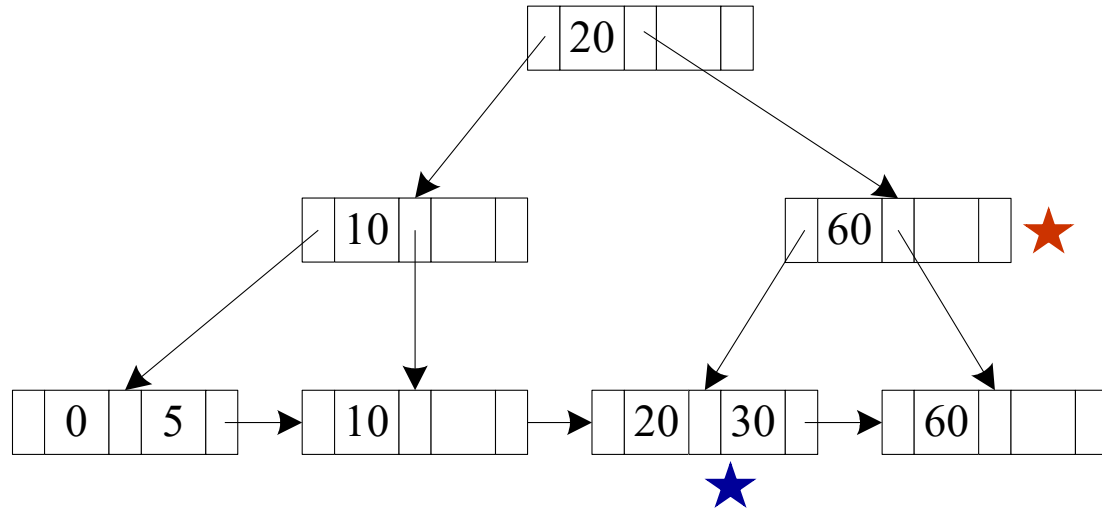


Example of B+- Tree: insert 30

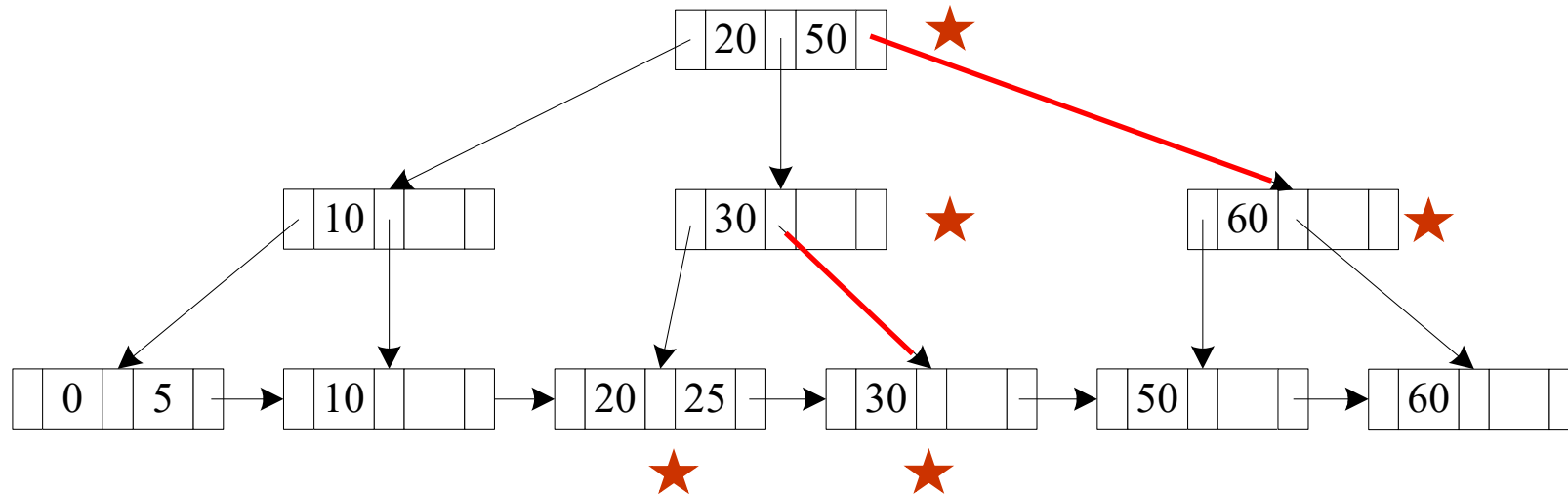
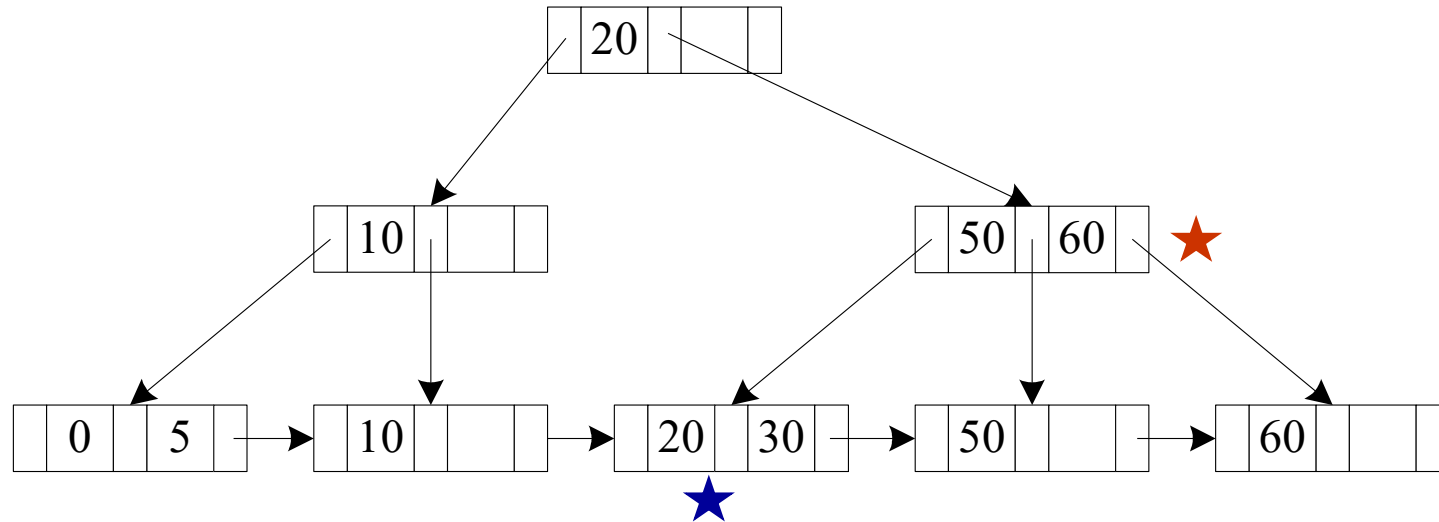
-----> 指向数据的指针
——> 指向节点的指针



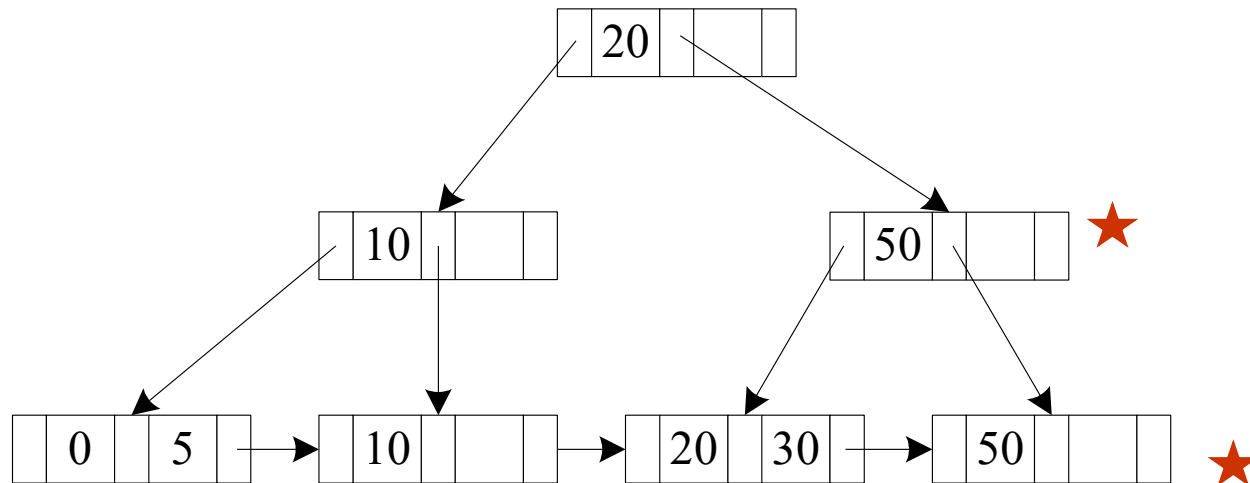
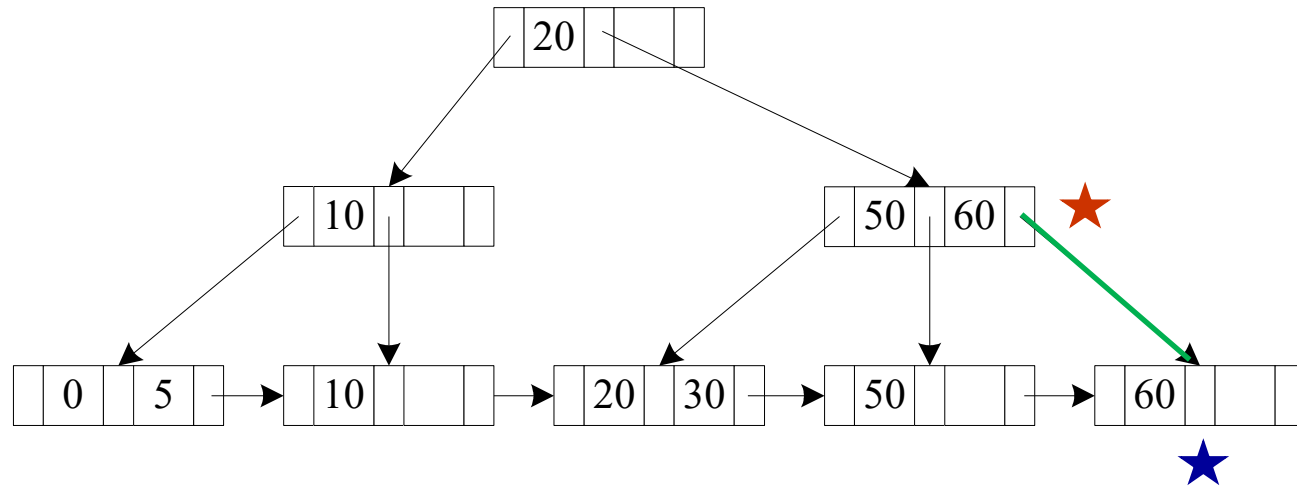
Example of B+- Tree: insert 50



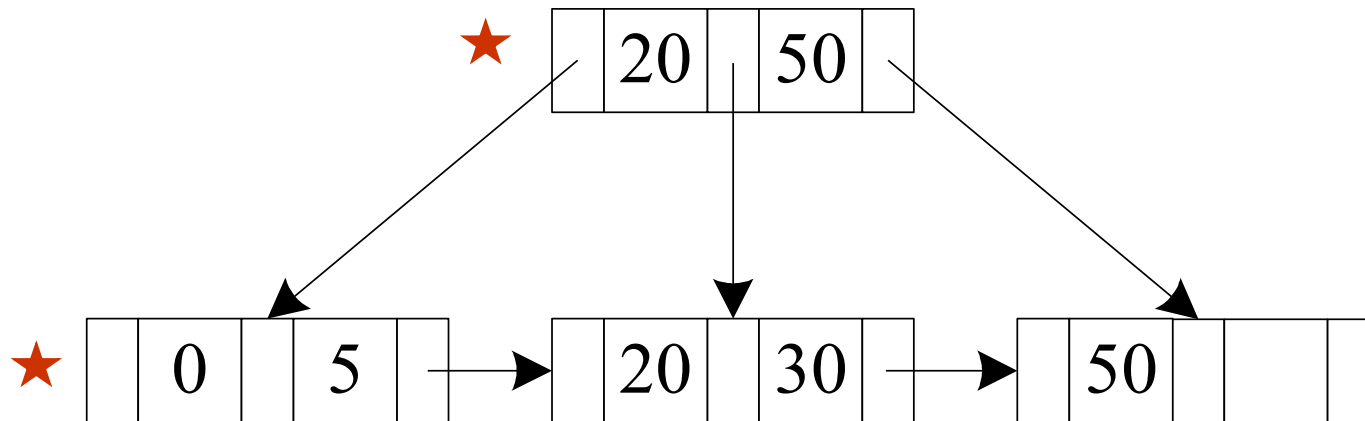
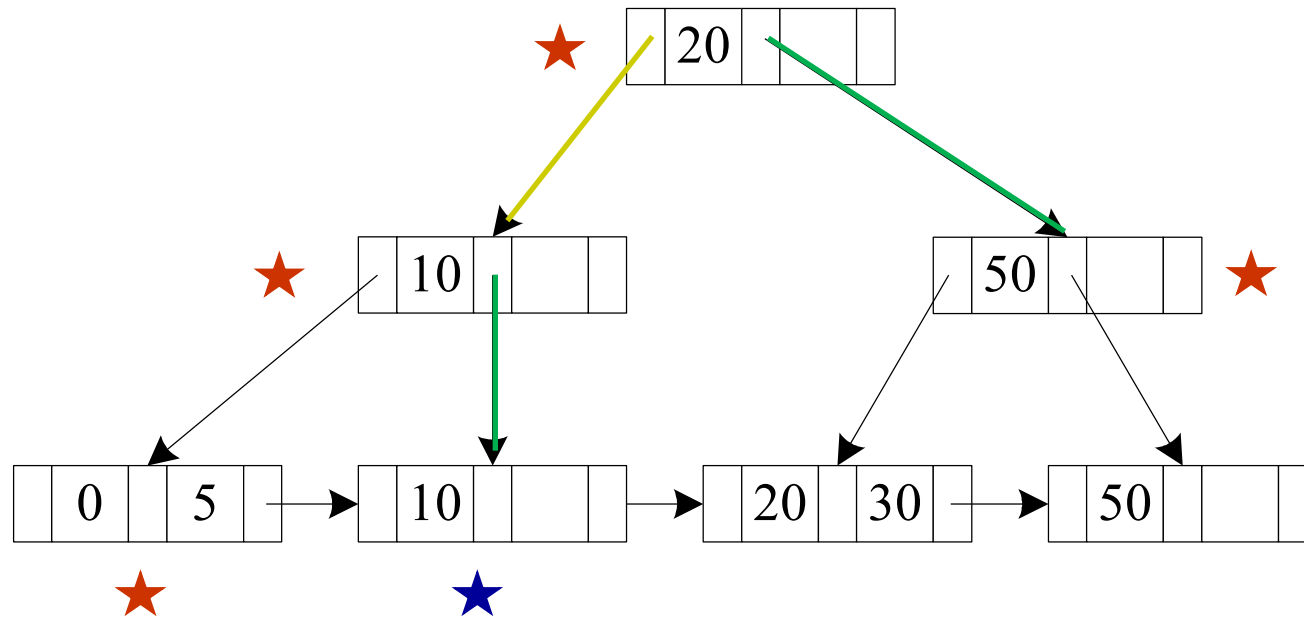
Example of B+- Tree: insert 25



Example of B+- Tree: delete 60



Example of B+- Tree: delete 10



B+- tree index : height and size estimation

person(*pid* **char(18)** **primary key**,
 name **char(8)**,
 age **smallint**,
 address **char(40)**);

block size : 4K

1000000 persons

person record size = 18+8+2+40 = 68

Records per block = 4096/68= 60.235 → 60

Number of the blocks for storing 1M persons= $\lceil 1000000/60 \rceil = 16667$

B+ tree n(fan-out) = $(4096-4)/(18+4) + 1 = 187$

$\lceil n/2 \rceil = 94$, inner pointers : 94~187

$\lceil n-1/2 \rceil = 93$, leaf values : 93~186

2 levels: min=2*93 =186 max= 187*186= **34,782**

3 levels: min=2*94*93 =17484 max= 187*187*186=6,504,234

4 levels: min=2*94*94*93=**1,643,496** max= 187*187*187*186= 1,216,291,758

Estimating height & size of B+-tree index

person(*pid* char(18), *name* char(8), *age* smallint,
address char(40), **primary key** (*pid*));

Block size : 4K

1000000 persons [*B+ tree size estimation.xlsx*](#)

2.641029	<=	B+ level	<=	3.8883			
3	<=	B+ level	<=	3			
		half full leaf nodes					
		leaf level nodes		10752.69	10752	nodes	
		second level nodes			114	nodes	
		root			1	node	
max		total nodes			10867		
		full leaf nodes					
		leaf level nodes		5376.344	5377	nodes	
		second level nodes			29	nodes	
		root			1	node	
min		total nodes			5407		

B⁺-Tree File Organization

B⁺-Tree File Organization:

Leaf nodes in a B⁺-tree file organization store **records**, instead of pointers

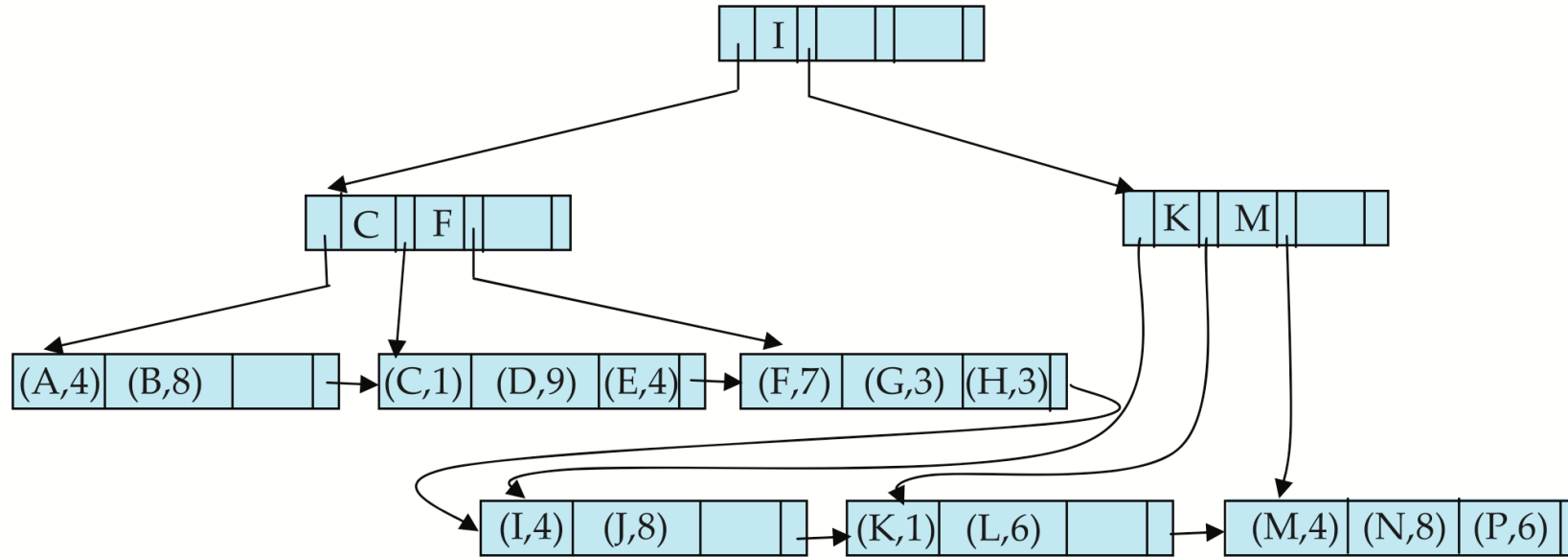
Helps keep data records clustered even when there are insertions/deletions/updates

Leaf nodes are still required to be half full

Since records are larger than pointers, the maximum number of records that can be stored in a leaf node is less than the number of pointers in a nonleaf node.

Insertion and deletion are handled in the same way as insertion and deletion of entries in a B⁺-tree index.

B⁺-Tree File Organization (Cont.)



Example of B⁺-tree File Organization

Good space utilization important since records use more space than pointers.

To improve space utilization, involve more sibling nodes in redistribution during splits and merges

Involving 2 siblings in redistribution (to avoid split / merge where possible) results in each node having at least $\lfloor 2n/3 \rfloor$ entries

B+- tree file organization : height and size estimation

```
person( pid          char(18) primary key,  
        name         char(8),  
        age          smallint,  
        address      char(40) );
```

block size : 4K

1000000 persons

person record size = 18+8+2+40 = 68

Max records per leaf node = (4096-4)/68 = 60.17 → 60

Half-full records per leaf node = 60/2 = 30

Min number of the leaf nodes for storing 1M persons = $\lceil 1000000/60 \rceil = 16667$

Max number of the leaf nodes for storing 1M persons = $\lceil 1000000/30 \rceil = 33333$

B+ tree n(fan-out) of non-leaf nodes = (4096-4)/(18+4) + 1 = 187

$\lceil n/2 \rceil = 94$, inner pointers : 94~187

*2 levels : min = 2*30=60 max = 187*60 = 11220*

3 levels: min=2*94*30 =5640 max= 187*187*60=**2,098,140**

4 levels: min=2*94*94*30=**530,160** max= 187*187*187*60 = **392,352,180**

5 levels: min=2*94*94*94*30=**49,835,040** max= 187*187*187*187*60= 7.34e10

Other Issues in Indexing

Record relocation and secondary indices

If a record **moves**, all secondary indices that store record pointers have to be updated

Node splits in B⁺-tree file organizations become very expensive

Solution: use primary-index search key instead of record pointer in secondary index

- ▶ Extra traversal of primary index to locate record
 - Higher cost for queries, but node splits are cheap
- ▶ Add record-id if primary-index search key is non-unique

Indexing Strings

Variable length strings as keys

Variable fanout

Use space utilization as criterion for splitting, not number of pointers

Prefix compression

Key values at internal nodes can be prefixes of full key

- ▶ Keep enough characters to distinguish entries in the subtrees separated by the key value
 - E.g. “Silas” and “Silberschatz” can be separated by “Silb”

Keys in leaf node can be compressed by sharing common prefixes

Multiple-Key Access

Use **multiple indices** for certain types of queries.

Example:

select *ID*

from *instructor*

where *dept_name* = "Finance" **and** *salary* = 80000

Possible strategies for processing query using indices on single attributes:

1. Use index on *dept_name* to find instructors with department name Finance; test *salary* = 80000
2. Use index on *salary* to find instructors with a salary of \$80000; test *dept_name* = "Finance".
3. Use *dept_name* index to find pointers to all records pertaining to the "Finance" department. Similarly use index on *salary*. Take intersection of both sets of pointers obtained.

Indices on Multiple Keys

Composite search keys are search keys containing more than one attribute

E.g. (*dept_name*, *salary*)

Lexicographic ordering: $(a_1, a_2) < (b_1, b_2)$ if either

$a_1 < b_1$, or

$a_1 = b_1$ and $a_2 < b_2$

Indices on Multiple Keys

Suppose we have an index on combined search-key
(*dept_name*, *salary*).

With the **where** clause

where *dept_name* = "Finance" **and** *salary* = 80000

the index on (*dept_name*, *salary*) can be used to fetch only records that satisfy both conditions.

Using separate indices is less efficient — we may fetch many records (or pointers) that satisfy only one of the conditions.

Can also efficiently handle

where *dept_name* = "Finance" **and** *salary* < 80000

But cannot efficiently handle

where *dept_name* < "Finance" **and** *balance* = 80000

May fetch many records that satisfy the first but not the second condition

Non-Unique Search Keys

If a search key a_i is not unique, create instead an index on a composite key (a_i, A_p) , which is unique

A_p could be a primary key, record ID, or any other attribute that guarantees uniqueness

Search for $a_i = v$ can be implemented by a range search on composite key, with range $(v, -\infty)$ to $(v, +\infty)$

Extra storage overhead for keys

Simpler code for insertion/deletion

More I/O operations are needed to fetch the actual records

If the index is clustering, all accesses are sequential

If the index is non-clustering, each record access may need an I/O operation

Widely used

Indexing in Main Memory

Random access in memory

Much **cheaper** than on disk/flash, but still **expensive** compared to cache read

Binary search for a key value within a large B⁺-tree node results in many cache misses

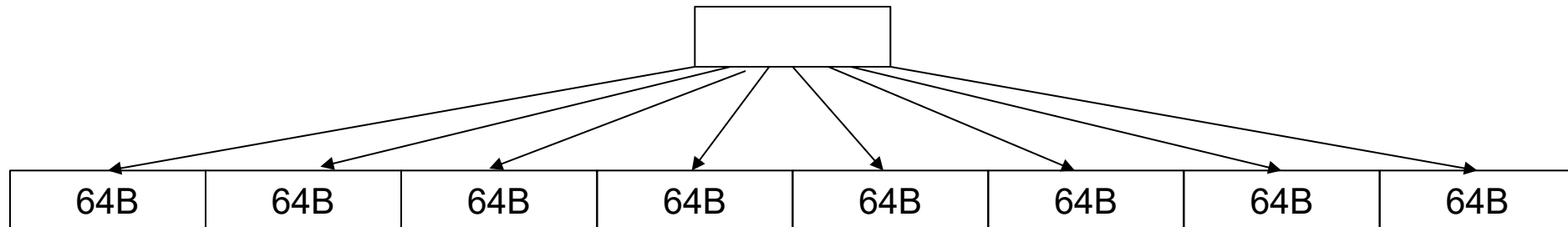
Data structures that **make best use of cache** preferable – **cache conscious**

B⁺- trees with **small nodes** that fit in cache line are preferable to reduce cache misses

Key idea:

use **large node size** to optimize **disk access**,

but structure data within a node using a tree with **small node size**, instead of using an array, to optimize **cache access**.



Bulk Loading and Bottom-Up Build

Inserting entries one-at-a-time into a B⁺-tree requires ≥ 1 IO per entry
assuming leaf level does not fit in memory
can be very inefficient for loading a large number of entries at a time
(**bulk loading**)

Efficient alternative 1: **Insert in sorted order**

sort entries first (using efficient external-memory sort algorithms)

insert in sorted order

- ▶ insertion will go to existing page (or cause a split)
- ▶ much improved IO performance, but most leaf nodes half full

Efficient alternative 2: **Bottom-up B⁺-tree construction**

As before sort entries

And then create tree layer-by-layer, starting with leaf level

Implemented as part of bulk-load utility by most database systems

Bottom-Up B+-tree Build

Bottom-up B+-tree construction

First sort index entries

Then create B+-tree layer-by-layer, starting with leaf level

The built B+-tree is written to disk using **sequential I/O** operations

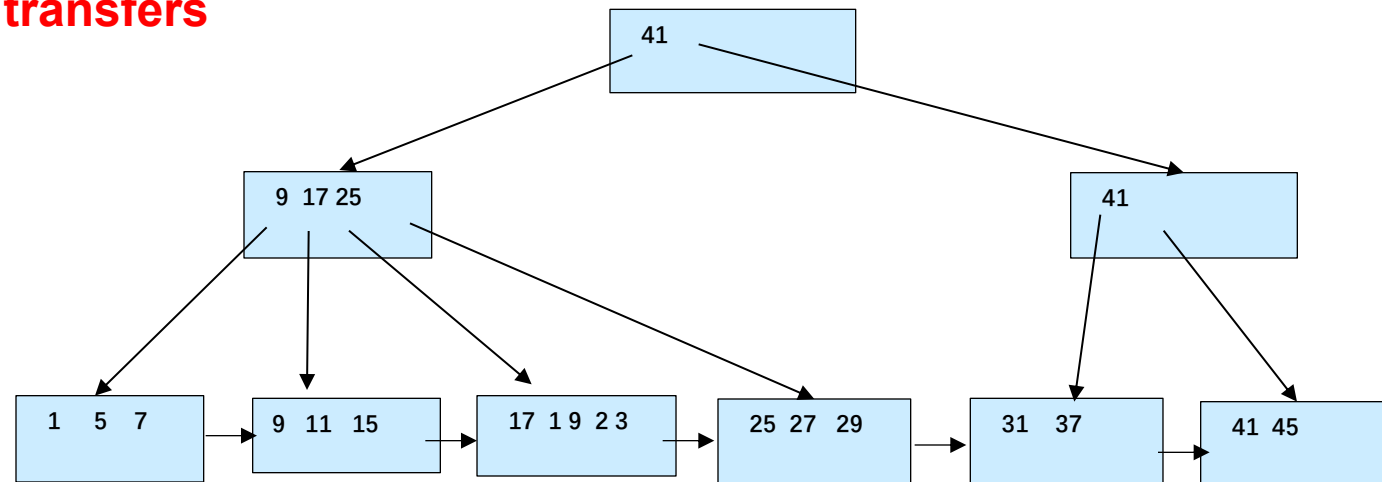
Example : build a B+-tree for the following 16 entries, assuming the fan-out(n) of the B+-tree is 4:

(23 25 27 29 1 5 7 9 11 31 37 41 45 15 17 19)

→ (1 5 7 9 11 15 17 19 23 25 27 29 31 37 41 45)

Estimated cost :

1 seek + 9 block transfers



Bottom-Up B+-tree Build

Bulk insert index entries

Build a new B+-tree using the Bottom-UP Build algorithm

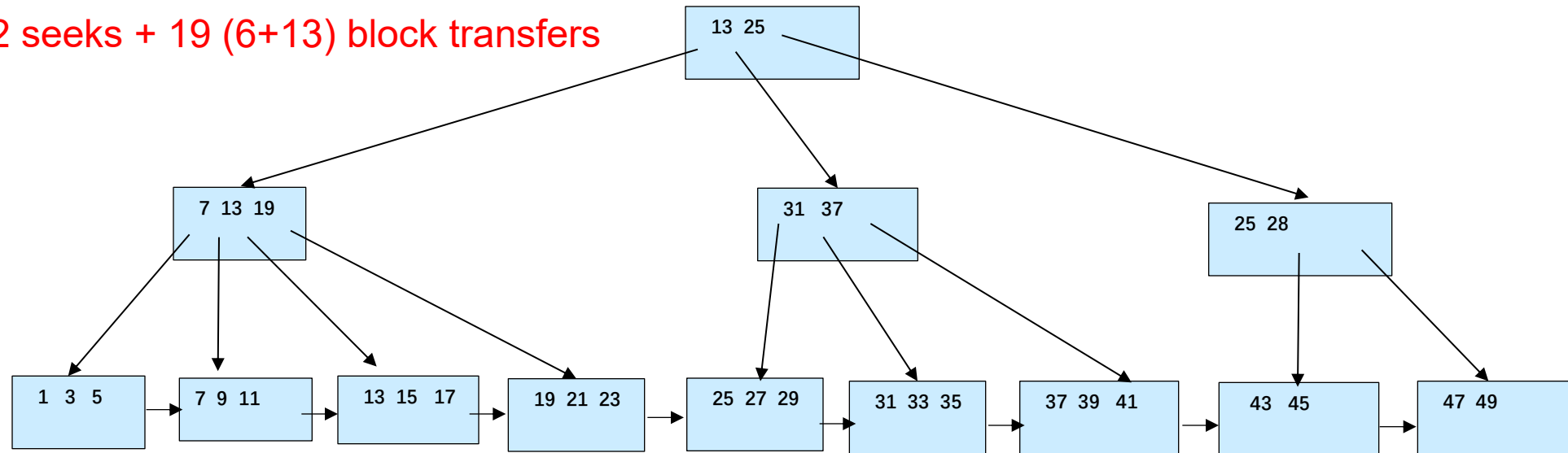
Example: insert following 9 entries into previous B+-tree

(21 33 35 39 43 47 49 3 13)

→ (3 13 21 33 35 39 43 47 49)

Estimated cost :

2 seeks + 19 (6+13) block transfers



Merge two existing two B+-trees , to create a new B+-tree using the Bottom-UP Build algorithm , as in **LSM-tree** Index.

Indexing on Flash

Random I/O cost much lower on flash

20 to 100 microseconds for read/write

Writes are not in-place, and (eventually) require a more expensive erase

Optimum page size therefore much smaller

Bulk-loading still useful since it minimizes page erases

Write-optimized tree structures (i.e., **LSM-tree**) have been adapted to minimize page writes for flash-optimized search trees

Write Optimized Indices

Performance of B⁺-trees can be poor for **write-intensive** workloads

- One I/O per leaf, assuming all internal nodes are in memory

- With magnetic disks, < 100 inserts per second per disk

- With flash memory, one page overwrite per insert

Two approaches to reducing cost of writes

- Log-structured merge tree (LSM-tree)**

- Buffer tree**

Log Structured Merge (LSM) Tree

Consider only inserts/queries for now

Records inserted first into in-memory tree (L_0 tree)

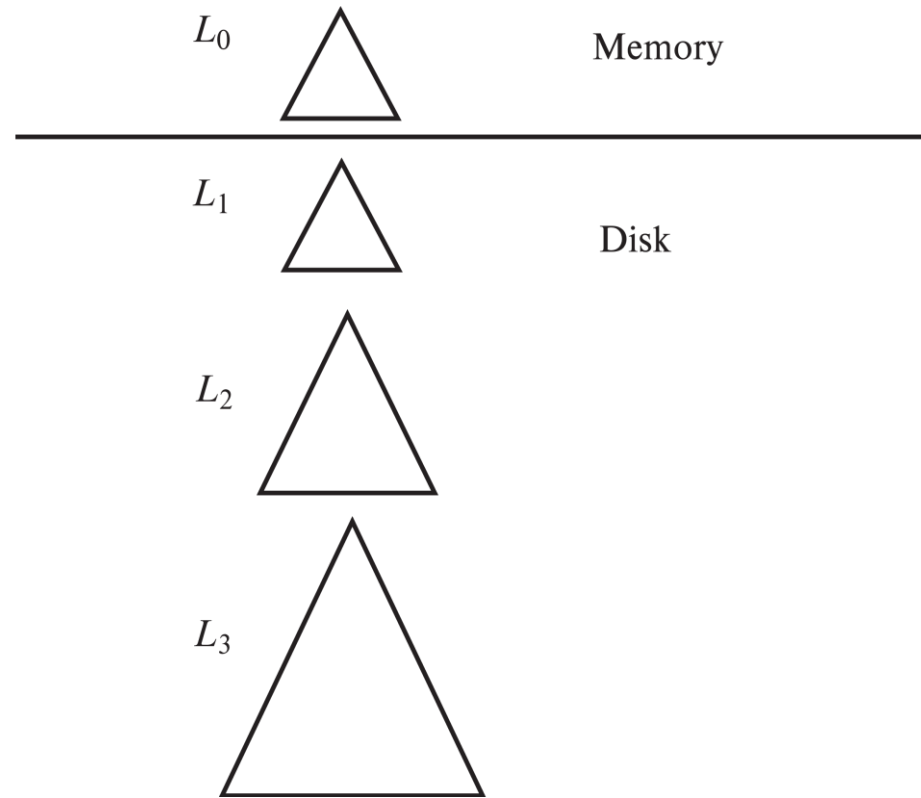
When in-memory tree is full, records moved to disk (L_1 tree)

B⁺-tree constructed using **bottom-up build** by merging existing L_1 tree with records from L_0 tree

When L_1 tree exceeds some threshold, merge into L_2 tree

And so on for more levels

Size threshold for L_{i+1} tree is **k times** size threshold for L_i tree



LSM Tree (Cont.)

Benefits of LSM approach

- Inserts are done using only **sequential I/O** operations

- Leaves are full**, avoiding space wastage

- Reduced number of I/O operations per record inserted as compared to normal B⁺-tree (up to some size)

Drawback of LSM approach

- Queries have to search multiple trees

- Entire content of each level copied multiple times

Stepped-merge index

- Variant of LSM tree with multiple trees at each level

- Reduces write cost compared to LSM tree

- But queries are even more expensive

- ▶ Bloom filters to avoid lookups in most trees

Details are covered in Chapter 24

LSM Tree (Cont.) - Stepped Merge Index

Stepped-merge index: variant of LSM tree with **k trees at each level** on disk

When all k indices exist at a level, merge them into one index of next level.

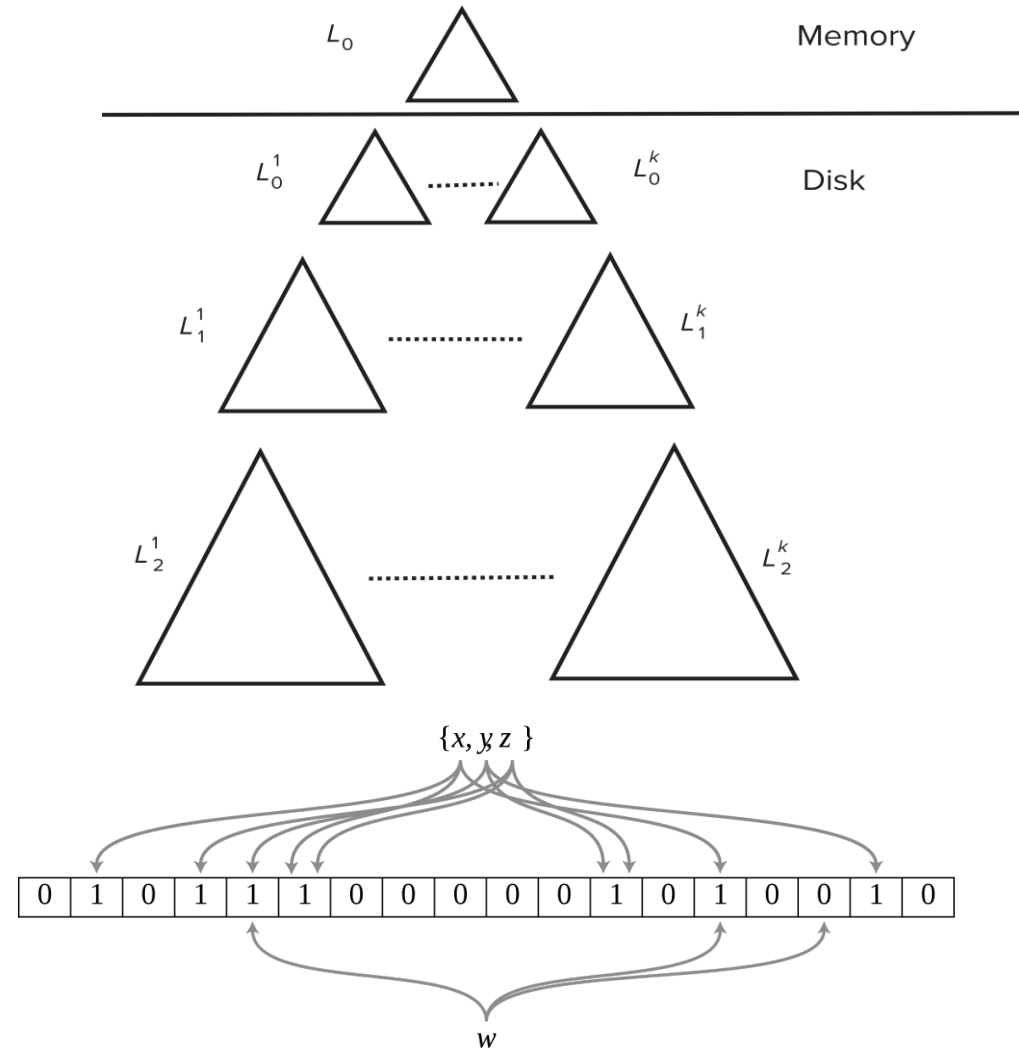
Reduces write cost compared to LSM tree

But queries are even more expensive since many trees need to be queried

Optimization for point lookups

Compute **Bloom filter** for each tree and store in-memory

Query a tree only if Bloom filter returns a positive result



LSM Trees (Cont.)

Deletion handled by adding special “delete” entries

Lookups will find both original entry and the delete entry, and must return only those entries that do not have matching delete entry

When trees are merged, if we find a delete entry matching an original entry, both are dropped.

Update handled using insert+delete

LSM trees were introduced for disk-based indices

But useful to minimize erases with flash-based indices

The stepped-merge variant of LSM trees is used in many BigData storage systems

- ▶ Google BigTable, Apache Cassandra, MongoDB
- ▶ And more recently in SQLite4, LevelDB, and MyRocks storage engine of MySQL

Buffer Tree

Key idea: each internal node of B⁺-tree has a buffer to store inserts

Inserts are moved to lower levels when buffer is full

With a large buffer, many records are moved to lower level each time

Per record I/O decreases correspondingly

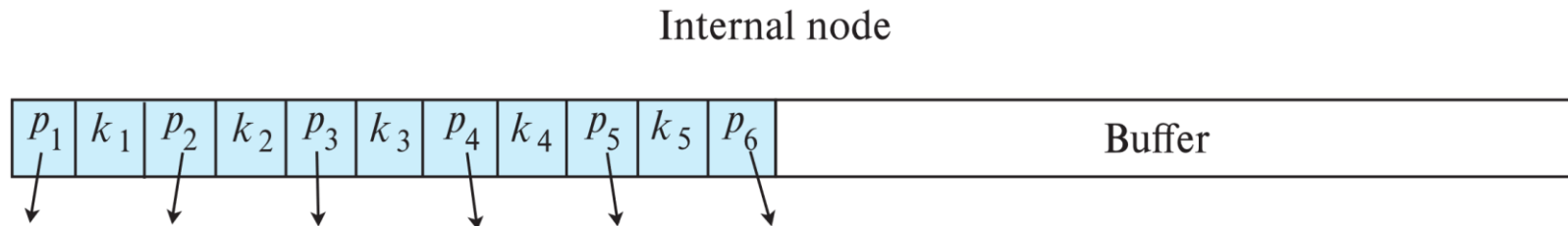
Benefits

Less overhead on queries

Can be used with any tree index structure

Used in PostgreSQL Generalized Search Tree (GiST) indices

Drawback: more random I/O than LSM tree



Bitmap Indices

Bitmap indices are a special type of index designed for efficient querying on multiple keys

Records in a relation are assumed to be numbered sequentially from, say, 0

Given a number n it must be easy to retrieve record n

- ▶ Particularly easy if records are of fixed size

Applicable on attributes that take on a relatively small number of distinct values

E.g., gender, country, state, ...

E.g., income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000- infinity)

A bitmap is simply an array of bits

Bitmap Indices (Cont.)

In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute

Bitmap has as many bits as records

In a bitmap for value v , the bit for a record is 1 if the record has the value v for the attribute, and is 0 otherwise

Example

record number				Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
	<i>ID</i>	<i>gender</i>	<i>income_level</i>	m	f		
0	76766	m	L1	10010	01101	L1	10100
1	22222	f	L2			L2	01000
2	12121	f	L1			L3	00001
3	15151	m	L4			L4	00010
4	58583	f	L3			L5	00000

Bitmap Indices (Cont.)

Bitmap indices are useful for queries on multiple attributes

not particularly useful for single attribute queries

Queries are answered using bitmap operations

Intersection (and)

Union (or)

Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap

E.g., $100110 \text{ AND } 110011 = 100010$

$100110 \text{ OR } 110011 = 110111$

$\text{NOT } 100110 = 011001$

Males with income level **L1**: $10010 \text{ AND } 10100 = 10000$

- ▶ Can then retrieve required tuples.
- ▶ Counting number of matching tuples is even faster

Bitmap Indices (Cont.)

Bitmap indices generally very small compared with relation size

E.g., if record is 100 bytes, space for a single bitmap is $1/800$ of space used by relation.

- ▶ If number of **distinct attribute values is 8**, bitmap is only **1%** of relation size

Efficient Implementation of Bitmap Operations

Bitmaps are packed into words; a single word and (a basic CPU instruction) computes and of 32 or 64 bits at once

E.g., 1-million-bit maps can be and-ed with just 31,250 instruction

Counting number of 1s can be done fast by a trick:

Use each byte to index into a precomputed array of 256 elements each storing the count of 1s in the binary representation

0	1	2	3	...	253	254	255
0	1	1	2		6	6	7

- ▶ Can use pairs of bytes to speed up further at a higher memory cost

Add up the retrieved counts

End of Chapter 14